

CEDRUS ^{α} : A Stateless Hash-based Signature Scheme

Siwei Sun, Zhen Qin, Fangyu Zheng, Zhiyu Zhang
siweisun.isaac@gmail.com

29/06/2026

Contents

Chapter 1 Specification	11
1.1 Notations and Preliminaries	11
1.1.1 Arrays, Byte Strings, and Integers	12
1.1.2 Encoding Function	12
1.2 CEDRUS ^α Parameters	14
1.3 Cryptographic Functions	15
1.4 The CEDRUS ^α Hypertree	15
1.5 Address Scheme	18
1.6 The One-time Signature (OTS) Scheme	21
1.6.1 OTS Public Key Generation	21
1.6.2 OTS Signature Generation	22
1.6.3 Computing an OTS Public Key From a Signature	22
1.7 eXtended Merkle Signature Scheme (XMSS)	23
1.7.1 Generating a Merkle Hash Tree	24
1.7.2 Generating an XMSS Signature	24
1.7.3 Computing an XMSS Public Key from a XMSS Signature	25
1.8 The CEDRUS ^α Hypertree Signature	27
1.8.1 Hypertree Signature Generation	27
1.8.2 Hypertree Signature Verification	27
1.9 The FORC Few-time Signature	27
1.9.1 Generating FORC Secret Values	29
1.9.2 Computing the Nodes of a FORC Tree	29
1.9.3 Generating a FORC Signature	30
1.9.4 Computing a FORC Public Key from a Signature	30
1.10 The CEDRUS ^α Signature Scheme	32
1.10.1 CEDRUS ^α Key Generation	32
1.10.2 CEDRUS ^α Signature Generation	34
1.10.3 Signature Verification	34
1.11 Parameter Sets and Instantiations	35
Chapter 2 Design Rationale	41
2.1 Notations and Preliminaries	41
2.2 The Inhomogeneous WOTS ^α One-time Signature	42
2.3 The CEDRUS ^α Hypertree	43

Chapter 3 Security Analysis	45
3.1 Security Reduction	45
3.2 Concrete Security	45
Chapter 4 Performance Analysis	47
4.1 Theoretical Performance Analysis	47
4.2 Implementations and Benchmarks	48
Chapter 5 Feature Overview	51
5.1 Techniques for Improving the SPHINCS ⁺ Framework	51
5.2 Advantages and Limitations	51
Bibliography	52

List of Figures

1.1	CEDRUS ^α hypertree	17
1.2	OTS hash address	18
1.3	OTS secret key generation address	18
1.4	OTS public key compression address	19
1.5	XMSS tree hash address	19
1.6	FTS tree hash address	19
1.7	FTS public key compression address	20
1.8	FORC secret key generation address	20
1.9	FORC hash chain address	20
1.10	Inhomogeneous WOTS ^α signature data format	22
1.11	XMSS signature data format for the i -th layer	24
1.12	Hypertree signature data format	27
1.13	FORC signature data format	29
1.14	CEDRUS ^α signature data format	34

List of Tables

1.1	CEDRUS ^{α} parameter sets	37
1.2	Definition of the functions for CEDRUS ^{α} -160 and CEDRUS ^{α} -256	38
1.3	Definition of the functions for CEDRUS ^{α} -384 CEDRUS ^{α} -512	39
4.1	A theoretical analysis of the performance of the proposed signature algorithms	49
4.2	Benchmark results for the proposed signature algorithms	49

List of Algorithms

1	<code>toInt(X, n)</code>	12
2	<code>toByte(x, n)</code>	13
3	$\rho_w^s : [C_s(\prod_{j=0}^{l-1}[w_j])] \rightarrow C_s(\prod_{j=0}^{l-1}[w_j])$	13
4	<code>chain($n, i, s, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	21
5	<code>pkGen_{OTS}($\mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	22
6	<code>Sign_{OTS}($n, \mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	23
7	<code>pkFromSig_{OTS}($\mathbf{SIG}_{\text{OTS}}, M, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	24
8	<code>xmssNode($\mathbf{SK.seed}, i, z, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	25
9	<code>Sign_{XMSS}($M, \mathbf{SK.seed}, \text{idx}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	25
10	<code>pkFromSig_{XMSS}($\text{idx}, \mathbf{SIG}_{\text{XMSS}}, M, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	26
11	<code>sign_{HT}($M, \mathbf{SK.seed}, \mathbf{PK.seed}, \text{idxTree}, \text{idxLeaf}$)</code>	28
12	<code>verify_{HT}($M, \mathbf{SIG}_{\text{HT}}, \mathbf{PK.seed}, \text{idxTree}, \text{idxLeaf}, \mathbf{PK.root}$)</code>	28
13	<code>skGenFORC($\mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}, \text{idx}$)</code>	29
14	<code>chainFORC($n, i, s, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	30
15	<code>nodeFORC($\mathbf{SK.seed}, i, z, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	31
16	<code>msgToIndices(md, a, ℓ, k)</code>	31
17	<code>signFORC($\text{md}, \mathbf{SK.seed}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	32
18	<code>pkFromSigFORC($\sigma_{\text{FORC}}, \text{md}, \mathbf{PK.seed}, \mathbf{ADRS}$)</code>	33
19	<code>keyGen_{SLH}($\mathbf{SK.seed}, \mathbf{SK.prf}, \mathbf{PK.seed}$)</code>	34
20	<code>signInternal_{SLH}($\text{msg}, \mathbf{SK}, \text{addrnd}$)</code>	35
21	<code>verifyInternal_{SLH}($\text{msg}, \mathbf{SIG}, \mathbf{PK}$)</code>	36

Chapter 1

Specification

This chapter specifies the stateless hash-based signature scheme **CEDRUS**^α, and provides instances meeting the security requirement made by the *Call for Proposals for the Next-generation Public-Key Cryptographic Algorithms* from the Institute of Commercial Cryptography Standards of China (NICCS) [1]. The call for proposal requires that the submitted algorithms should provide parameter sets achieving the following security levels:

- Category I: 128-bit classical security and 80-bit quantum security.
- Category II: 256-bit classical security and 128-bit quantum security.
- Category III: 512-bit classical security and 256-bit quantum security.

Parameter sets with 384-bit classical security and 192-bit quantum security is optional. In this chapter, we provides parameter sets for all the security levels listed above, including the optional one. Note that for category I, our parameter set achieves 160-bit classical security and 80-bit quantum security.

1.1 Notations and Preliminaries

For a finite set $\mathbb{G}$, the number of elements in $\mathbb{G}$ is denoted by $|\mathbb{G}|$. We use $\mathbb{Z}_+$ to denote the set of all positive integers and let $\mathbb{N} = \mathbb{Z}_+ \cup \{0\}$. The set of real numbers is denoted by $\mathbb{R}$. Let $\mathbb{F}_2 = \{0, 1\}$ be the binary field and $\mathbb{B} = \mathbb{F}_2^8$ be the set of all 8-bit strings (bytes). For a positive integer w , $[w]$ is defined to be the set $\{0, \dots, w-1\}$. The Cartesian product of two sets A and B is represented by $A \times B$. For $\mathbf{w} = (w_0, \dots, w_{l-1}) \in \mathbb{Z}_+^l$, the set $[w_0] \times \dots \times [w_{l-1}]$ is abbreviated as $\prod_{j=0}^{l-1} [w_j]$. When $w_0 = \dots = w_{l-1} = w$, $\prod_{j=0}^{l-1} [w_j] = [w]^l$. For $i \in \mathbb{N}$ and $m \in \mathbb{Z}_+$, $\text{toBin}(i, m)$ is the m -bit binary representation of i . For example, $\text{toBin}(5, 8) = 00000101$. Let x be a binary string, $\text{Int}(x)$ denotes the unsigned integer it represents, $\text{leadingBits}(x, n)$ denotes the most significant n bits and $\text{trailingBits}(x, n)$ denotes the least significant n bits of x .

To simplify the presentation, we introduce the helper function $\text{IntSeq}_{\mathbf{w}}(\cdot)$, which converts a binary string or an integer to a sequence of integers. Let $\mathbf{w} = (w_0, \dots, w_{v-1}) \in \mathbb{Z}_+^v$ and s be a $(\log_2 w_0 + \dots + \log_2 w_{v-1})$ -bit binary string. Then, $\text{IntSeq}_{\mathbf{w}}(s) = (i_0, \dots, i_{v-1})$, where i_t for $0 \leq t < v$ is the non-negative integer represented by the $\log_2 w_t$ -bit substring formed by the $((\sum_{j=0}^t \log_2 w_j) - \log_2 w_t)$ -th bit to $((\sum_{j=0}^t \log_2 w_j) - 1)$ -th bit segment of s . When the input is a non-negative integer $k \in \{0, \dots, 2^{\log_2 w_0 + \dots + \log_2 w_{v-1}} - 1\}$, $\text{IntSeq}_{\mathbf{w}}(k)$ should be understood as $\text{IntSeq}_{\mathbf{w}}(\text{toBin}(k, \log_2 w_0 + \dots + \log_2 w_{v-1}))$.

Example 1. Let $s = 101011$ be a 6-bit binary string, $k = 27$ be an integer whose binary representation is 11011 , and $\mathbf{w} = (4, 8, 2)$. Then, $\text{IntSeq}_{\mathbf{w}}(s) = (2, 5, 1)$, and $\text{IntSeq}_{\mathbf{w}}(k) = \text{IntSeq}_{\mathbf{w}}(\text{toBin}(k, 6)) = \text{IntSeq}_{\mathbf{w}}(011011) = (1, 5, 1)$.

For $a \in \mathbb{N}$ and $b \in \mathbb{Z}_+$, $a \bmod b$ denotes the integer i in $\{0, \dots, b-1\}$ such that $a - i$ is divisible by b . In addition, $a \text{ quo } b$ denotes the integer $j = \frac{a-i}{b}$.

Let $l, w_0, \dots, w_{l-1}$ be positive integers. For $s \in [1 + \sum_{j=0}^{l-1} (w_j - 1)]$, we define $\mathcal{C}_s(\prod_{j=0}^{l-1} [w_j])$ to be the set $\{(v_0, \dots, v_{l-1}) \in \prod_{j=0}^{l-1} [w_j] : \sum_{j=0}^{l-1} v_j = s\}$.

1.1.1 Arrays, Byte Strings, and Integers

If X is a byte string of length n , then $X[i]$ for $i \in \{0, \dots, n-1\}$ will refer to the i -th element in the string X . If X is an array of m n -byte strings, then $X[i]$ for $i \in \{0, \dots, m-1\}$ will refer to the i -th n -byte string in X , and X will refer to the $m \cdot n$ -byte string $X[0] \parallel X[1] \parallel \dots \parallel X[m-1]$. A byte string may be interpreted as the big-endian representation of an integer. In such cases, a byte string X of length n is converted to the integer

$$X[0] \cdot 256^{n-1} + X[1] \cdot 256^{n-2} + \dots + X[n-2] \cdot 256 + X[n-1]. \quad (1.1)$$

Similarly, an integer x may be converted to a byte string of length n by finding coefficients $x_0, x_1, \dots, x_{n-1} \in \{0, \dots, 255\}$ such that

$$x = x_0 \cdot 256^{n-1} + x_1 \cdot 256^{n-2} + \dots + x_{n-2} \cdot 256 + x_{n-1} \quad (1.2)$$

and then setting the byte string to be $x_0 \parallel \dots \parallel x_{n-1}$. Algorithm 1 is a function that converts a byte string X of length n to an integer, and Algorithm 2 is a function that converts an integer x to a byte string of length n .

Algorithm 1: $\text{toInt}(X, n)$

Input: An n -byte string X

Output: Integer value of X

1 $total \leftarrow 0$

2 **for** i from 0 to $n-1$ **do**

3 $total \leftarrow 256 \cdot total + X[i]$

4 **return** $total$

1.1.2 Encoding Function

We present an injective encoding from $\mathbb{F}_2^{8n}$ to $\mathcal{C}_s(\prod_{j=0}^{l-1} [w_j])$, which will be employed by our inhomogeneous WOTS $^\alpha$ one-time signature scheme described in Section 1.6. First, we establish a one-to-one mapping $\rho_{\mathbf{w}}^s : [|\mathcal{C}_s(\prod_{j=0}^{l-1} [w_j])|] \rightarrow \mathcal{C}_s(\prod_{j=0}^{l-1} [w_j])$ for $\mathbf{w} = (w_0, \dots, w_{l-1}) \in \mathbb{Z}_+^l$ and $0 \leq s \leq \left\lfloor \frac{\sum_{i=0}^{l-1} (w_i - 1)}{2} \right\rfloor$.

Lemma 1. Let l be a positive integer, $\mathbf{w} = (w_0, \dots, w_{l-1}) \in \mathbb{Z}_+^l$, and $0 \leq s \leq \left\lfloor \frac{\sum_{i=0}^{l-1} (w_i - 1)}{2} \right\rfloor$. The mapping $\rho_{\mathbf{w}}^s$ implemented by Algorithm 3 is bijective.

Algorithm 2: $\text{toByte}(x, n)$

Input: In integers x and n **Output:** An array of n bytes containing binary representation of x in big-endian byte-order

```

1  $total \leftarrow x$ 
2 for  $i$  from 0 to  $n - 1$  do
3    $S[n - 1 - i] \leftarrow total \bmod 256$ 
4    $total \leftarrow (total \gg 8)$ 
5 return  $S$ 

```

Proof. See [2]. □

Algorithm 3: $\rho_{\mathbf{w}}^s : [|\mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])|] \rightarrow \mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])$

Input: $x \in [|\mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])|]$ **Output:** $y = \rho_{\mathbf{w}}^s(x) \in \mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])$

```

1  $x \leftarrow x$ 
2  $s \leftarrow s$ 
3 for  $i$  from  $l$  to 1 do
4   for  $u$  from 0 to  $\min(w_{i-1} - 1, s)$  do
5     if  $x \geq \zeta_{\mathbf{w}}(i - 1, s - u)$  then
6        $x \leftarrow x - \zeta_{\mathbf{w}}(i - 1, s - u)$ 
7     else
8        $y_{l-i} \leftarrow u$ 
9       break
10   $s \leftarrow s - y_{l-i}$ 
11 return  $(y_0, \dots, y_{l-1})$ 

```

At this point, it is quite straightforward to build an injective encoding $\tau_{\mathbf{w}}^s : \mathbb{F}_2^{8n} \rightarrow \mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])$ with $\rho_{\mathbf{w}}^s$ when $|\mathcal{C}_s(\prod_{j=0}^{l-1}[w_j])| \geq |\mathbb{F}_2^{8n}|$. In this encoding, we simply map $x \in \mathbb{F}_2^{8n}$ to $\tau_{\mathbf{w}}^s(x) = \rho_{\mathbf{w}}^s(\text{Int}(x))$.

1.2 CEDRUS^α Parameters

In our design, the following parameters are used.

- n : The security strength of the signature is $8n$ -bit with respect to classical attacks. The size of a node of an **XMSS** tree is n -byte. The node of a hash chain is n -byte. The size of a node of a **PORS** tree is n -byte.
- $\mathbf{len}$: The hash-chain based one-time signature contains $\mathbf{len}$ chains.
- $\mathbf{w} = (w_0, \dots, w_{\mathbf{len}-1})$: The generalized Winternitz parameter. For $0 \leq i < \mathbf{len}$, the i -th chain of the one-time signature has w_i nodes, indexed by $\{0, \dots, w_i - 1\}$.
- a : There are k subtrees for the **FORC** few-time signature with height a .
- w' : There are w' nodes in a hash chain connecting the a secret value to the leaf node of a **FORC** tree (including the end nodes).
- k : There are k subtrees for the **FORC** few-time signature with height a .
- d : There are d layers of **XMSS** trees in the virtual hypertree.
- h : The total height of the virtual hypertree is h .

When the above parameters are given, we can determine the values of the following parameters.

- $\mathbf{h} = (h_0, \dots, h_{d-1})$: The height of the **XMSS** tree in layer i of the virtual hypertree is h_i , and $h_0 + \dots + h_{d-1} = h$.
- m : To sign a message **msg**, we first hash it to an m -byte digest with

$$m = \left\lceil \frac{k \cdot a}{8} \right\rceil + \left\lceil \frac{h - h_0}{8} \right\rceil + \left\lceil \frac{h_0}{8} \right\rceil.$$

1.3 Cryptographic Functions

We employ the following functions in our design:

$$\left\{ \begin{array}{l} \mathbf{F} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^n \rightarrow \mathbb{B}^n \\ \mathbf{H} : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{2n} \rightarrow \mathbb{B}^n \\ \mathbf{T}_l : \mathbb{B}^n \times \mathbb{B}^{32} \times \mathbb{B}^{l \cdot n} \rightarrow \mathbb{B}^n, \quad l \in \{\text{len}, k\} \\ \mathbf{PRF}_{\text{MSG}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \rightarrow \mathbb{B}^n \\ \mathbf{H}_{\text{MSG}} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^* \times \mathbb{B}^{n_{\text{ctr}}^{\text{FTS}}} \rightarrow \mathbb{B}^m \\ \mathbf{PRF} : \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^{32} \rightarrow \mathbb{B}^n \end{array} \right. \quad (1.3)$$

$\mathbf{F}$ is used to compute the nodes in the hash chains corresponding to the one-time signatures (see Line 5 of Algorithm 4) and the leaves of the FORC trees (see Line 7 of Algorithm 15). $\mathbf{H}$ is used to compute the non-leaf nodes of XMSS trees and FORC trees (see Line 11 of Algorithm 8 and Line 16 of Algorithm 15). $\mathbf{T}_{\text{len}}$ is used to compute the compression value of the end nodes of the hash chains for a one-time signature (see Line 13 of Algorithm 5), which is also a leaf node of an XMSS tree (see Line 4 of Algorithm 8). $\mathbf{T}_k$ is used to compute the compression value of the roots of the subtrees of the FORC tree (see Line 13 of Algorithm 5). $\mathbf{H}_{\text{MSG}}$ is used to compute a digest for a message during the randomized Hash-and-Sign paradigm (see Line 4 of Algorithm 20). $\mathbf{PRF}_{\text{MSG}}$ is used to compute a random input value for $\mathbf{H}_{\text{MSG}}$ (see Line 3 of Algorithm 20). $\mathbf{PRF}$ is used to generate the secret preimages of the one-time signature (see Line 7 of Algorithm 5) and the secret preimages of the FORC few-time signature (see Line 6 of Algorithm 13). $\mathbf{F}$, $\mathbf{H}$, $\mathbf{T}_l$, and $\mathbf{PRF}$ take a 32-byte data structure called address (the format of the address is specified in Section 1.5).

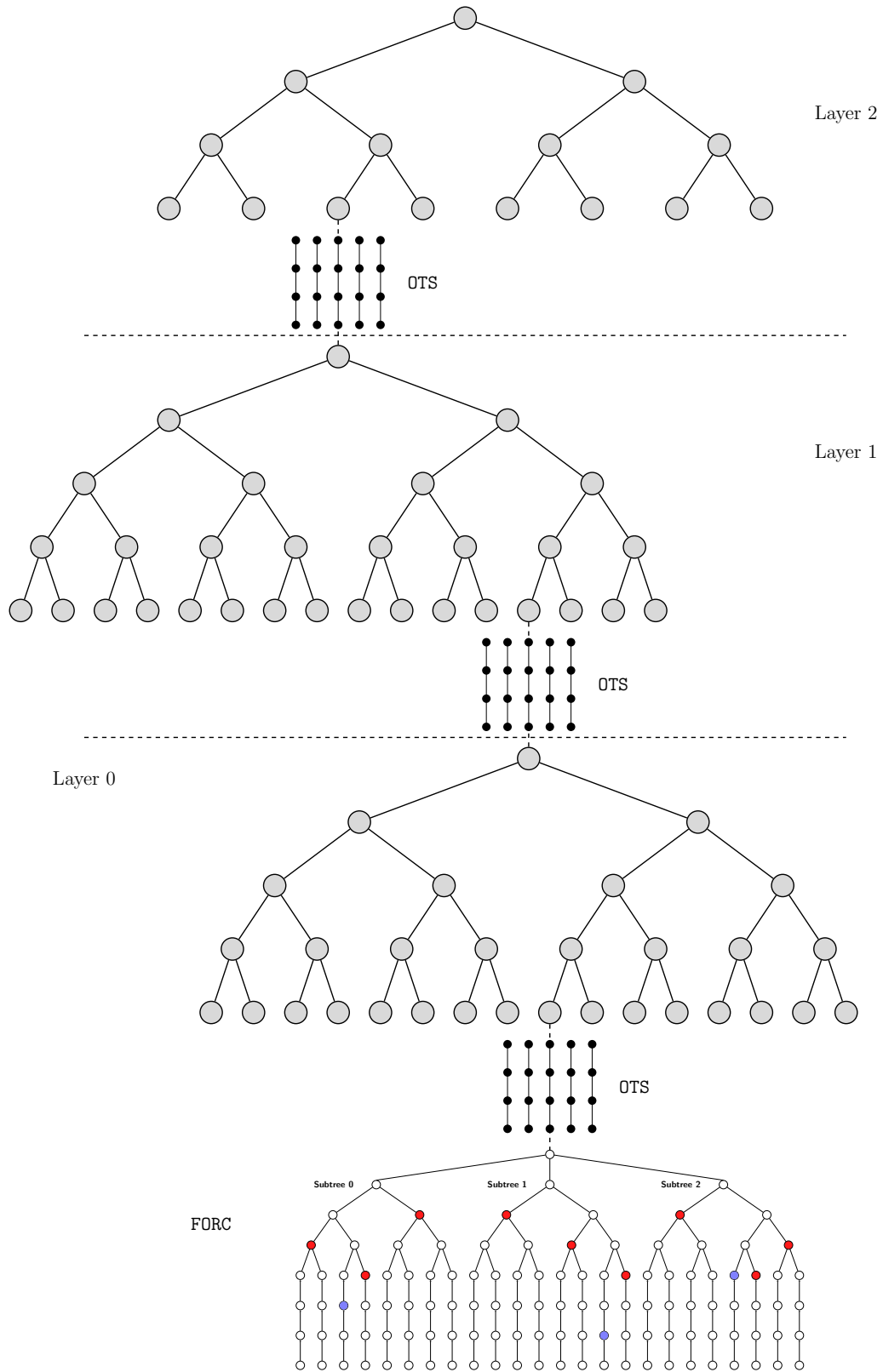
1.4 The CEDRUS^α Hypertree

The CEDRUS^α signature scheme can be described by a virtual hypertree as shown in Figure 1.1. The hypertree contains d layers. Layer 0 is the bottom layer and layer $d - 1$ is the top layer. In each layer there are certain number of perfect binary trees with the same height called XMSS trees. Let $h = h_0 + \dots + h_{d-1}$ be the total height of the hypertree. In layer 0, there are 2^{h-h_0} XMSS trees with height h_0 . In layer i , there are $2^{h-\sum_{u=0}^i h_u}$ XMSS trees with height h_i . In layer $d - 1$ (the top layer), there are exactly one XMSS tree with height h_{d-1} . Each XMSS tree with height h_i has 2^{h_i} leaves. Each leaf node corresponds to an instance of a one-time signature. Each leaf node in the bottom layer is associated with a FORS tree, and thus there are 2^h FORS trees. Each FORS tree contains k subtrees with height a . In the hypertree, each XMSS tree in layer i is associated with a leaf node of an XMSS tree in layer $i + 1$. Specifically, for $0 \leq i < d - 1$, the j -th XMSS tree $\Phi_j^{(i)}$ in layer i is associated with the $(j \bmod 2^{h_{i+1}})$ -th leaf node of the $(j \text{ quo } 2^{h_{i+1}})$ -th XMSS tree in layer $i + 1$.

The secret key of CEDRUS^α is $(\mathbf{SK.seed}, \mathbf{SK.prf}, \mathbf{PK.root}, \mathbf{PK.seed}) \in \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n \times \mathbb{B}^n$, and the public key is $(\mathbf{PK.root}, \mathbf{PK.seed}) \in \mathbb{B}^n \times \mathbb{B}^n$, where $\mathbf{SK.seed}$ and $\mathbf{SK.prf}$ are private, $\mathbf{PK.root}$ is the root of the XMSS tree in the top layer, and $\mathbf{PK.seed}$ is public. To sign a message $\text{msg} \in \mathbb{F}_2^*$, we first compute $\mathbf{R} = \mathbf{PRF}_{\text{MSG}}(\mathbf{SK.prf}, \text{optRand}, \text{msg})$. Then, we hash the message to obtain the

$$\text{digest} = \mathbf{H}_{\text{MSG}}(\mathbf{R}, \mathbf{PK.seed}, \mathbf{PK.root}, \text{msg}),$$

where `optRand` is `PK.seed` or a fresh random value. For the former case, `CEDRUSα` is a deterministic signature. Then, we divide the m -byte `digest` into 3 segments. The size of the 0th segment is $\lceil \frac{k \cdot a}{8} \rceil$ -byte, the size of the 1st segment is $\lceil \frac{h-h_0}{8} \rceil$ -byte, and the size of the 2nd segment is $\lceil \frac{h_0}{8} \rceil$ -byte. The 1st segment is used to select one `XMSS` tree in the bottom layer, and the 2nd segment is used to select one leaf of the selected `XMSS` tree. Let `md` be the 1st segment. Let the selected leaf node be the `idxLeaf`-th leaf of the `idxTree`-th `XMSS` tree in the bottom layer. Denote the j -th `XMSS` tree in layer i by $\Phi_j^{(i)}$. We first sign `md` with the `FORS` instance associated with the t_0 -th leaf of the `XMSS` tree $\Phi_{j_0}^{(0)}$, where $j_0 = \text{idxTree}$, $t_0 = \text{idxLeaf}$. Next, we sign the public key of the `FORS` instance with the OTS corresponding to the t_0 -th leaf node of $\Phi_{j_0}^{(0)}$. Then, we sign the root of $\Phi_{j_0}^{(0)}$ with the OTS corresponding to the t_1 -th leaf node of $\Phi_{j_1}^{(1)}$, $\dots$, and sign the root of $\Phi_{j_{d-3}}^{(d-3)}$ with the OTS corresponding to the t_{d-2} -th leaf node of $\Phi_{j_{d-2}}^{(d-2)}$. Finally, we sign the root of $\Phi_{j_{d-2}}^{(d-2)}$ with the OTS corresponding to the t_{d-1} -th leaf node of $\Phi_{j_{d-1}}^{(d-1)}$, where $j_{k+1} = j_k \bmod 2^{h_{k+1}}$, and $t_{k+1} = j_k \bmod 2^{h_{k+1}}$ ($0 \leq k < d$).

Figure 1.1: CEDRUS^α hypertree

1.5 Address Scheme

The functions **F**, **H**, **PRF** and **T_l** for $l \in \{\text{len}, k\}$ described in Section 1.3 take a 32-byte address **ADRS** as input. An **ADRS** consists of public values indicating the position of the data element within the virtual hypertree being computed by the functions. A different **ADRS** value is used for each call to each function. The structure of an **ADRS** conforms to word boundaries, with each word being 4 bytes long and the values of the data fields in **ADRS** are encoded as unsigned integers. There are 7 different types of addresses. These 7 types of **ADRS** share 3 common fields, including **layerAddr**, **treeAddr** and **addrType**. **layerAddr** $\in \{0, \dots, d-1\}$ takes 4 bytes, encoding the layer number of the data element being computed, with 0th layer being the bottom layer and $(d-1)$ -th layer being the top layer. **treeAddr** takes 12 bytes which indicates the index of the **XMSS** tree in a specific layer. If **layerAddr** = i for an address data structure, then the range of its **treeAddr** is $\{0, 1, \dots, 2^{h-\sum_{j=0}^i h_j} - 1\}$. **addrType** takes 4 bytes, indicating the type of the address, whose range is $\{\text{OTS_HASH} = 0x00, \text{OTS_PK} = 0x01, \text{XMSS_TREE} = 0x02, \text{FORS_TREE} = 0x03, \text{FORS_ROOT} = 0x04, \text{OTS_PRF} = 0x05, \text{FTS_PRF} = 0x06, \text{FTS_CHAIN} = 0x07\}$, where **OTS_HASH** = 0x00 corresponds to an **OTS** hash address (see Figure 1.2), **OTS_PRF** = 0x05 corresponds to an **OTS** secret key generation address (see Figure 1.3), **OTS_PK** = 0x01 corresponds to an **OTS** public key compression address (see Figure 1.4), **XMSS_TREE** = 0x02 corresponds to an **XMSS** tree hash address (see Figure 1.5), **FORS_TREE** = 0x03 corresponds to an **FTS** tree hash address (see Figure 1.6), **FTS_ROOT** = 0x04 corresponds to an **FTS** public key compression address (see Figure 1.7), **FTS_PRF** = 0x06 corresponds to an **FTS** secret key generation address (see Figure 1.8), and **FTS_CHAIN** = 0x07 corresponds to an **FTS** hash chain address (see Figure 1.9).

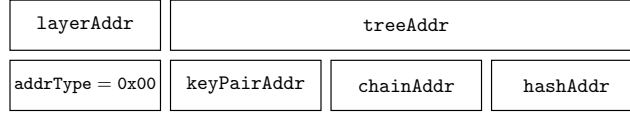


Figure 1.2: **OTS** hash address

As shown in Figure 1.2, **addrType** = **OTS_HASH** = 0x00 for an **OTS** hash address. An **OTS** hash address is used as input to **F** for computing the hash chains of **OTS** signatures (see Line 1 and Line 5 of Algorithm 4). Let **layerAddr** = i . The value of **keyPairAddr** indicates the index of the leaf of an **XMSS** tree with which the **OTS** instance concerned is associated, and its range is $\{0, 1, \dots, 2^{h_i} - 1\}$. The value of **chainAddr** determines the index of the hash chain being computed, whose range is $\{0, 1, \dots, \text{len} - 1\}$. Let **chainAddr** = j . **hashAddr** indicated the relative position of a node being computed within a specific hash chain, whose range is $\{0, 1, \dots, w_j - 2\}$.

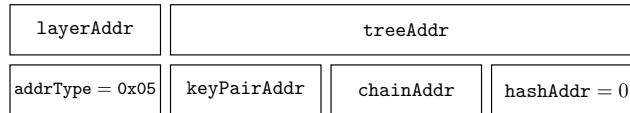


Figure 1.3: **OTS** secret key generation address

As shown in Figure 1.3, **addrType** = **OTS_PRF** = 0x05 for an **OTS** secret key generation address. **PRF** takes an **OTS** secret key generation address as input when computing the array of secret preimage elements of an **OTS** instance based on the secret seed **SK.seed** of the **CEDRUS^α** secret key (see Line 3 and Line 7 of Algorithm 5). Let **layerAddr** = i . For

an OTS secret key generation address, the value of **keyPairAddr** indicates the index of the leaf of an XMSS tree with which the OTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_i} - 1\}$. The value of **chainAddr** determines the index of the hash chain being computed, whose range is $\{0, 1, \dots, \text{len} - 1\}$. The value of **hashAddr** is always set to 0 for an OTS secret key generation address.

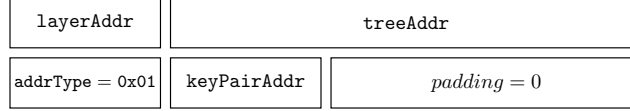


Figure 1.4: OTS public key compression address

As shown in Figure 1.4, **addrType** = OTS.PK = 0x01 for an OTS public key compression address. **T_{len}** takes an OTS public key compression address as input when compressing the end nodes of the hash chains of an OTS instance to an n -byte value, which corresponds to a leaf of an XMSS tree (see Line 11 and Line 13 of Algorithm 5). Let **layerAddr** = i . For an OTS public key compression address, the value of **keyPairAddr** indicates the index of the leaf of an XMSS tree with which the OTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_i} - 1\}$. The last 8 bytes of the address ADRS are set to 0.

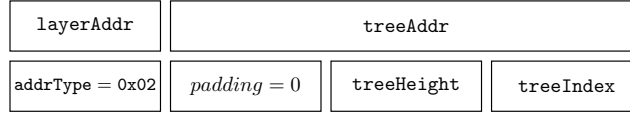


Figure 1.5: XMSS tree hash address

As shown in Figure 1.5, **addrType** = XMSS.TREE = 0x02 for an XMSS tree hash address. To construct the XMSS trees in the CEDRUS^α hypertree, **H** takes an XMSS tree hash address as input (see Line 8 and Line 11 of Algorithm 8). Let **layerAddr** = i . For an XMSS tree hash address, the value of **treeHeight** indicates the height of the nodes being computed by **H** within an XMSS tree, whose range is $\{1, \dots, h_i\}$. The height of the root node is h_i . The nodes with height 0 (i.e., the leaves) are not computed by **H**. Instead, they are computed with **F**. The value of **treeIndex** determines the position of the node being computed within the h -th level (formed by the nodes with height h) of an XMSS tree, whose range is $\{0, \dots, 2^{h_i-h} - 1\}$.

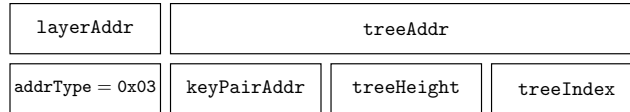


Figure 1.6: FTS tree hash address

As shown in Figure 1.6, **addrType** = FTS.TREE = 0x03 for an FORC tree hash address. **H** or **F** takes an FORC tree hash address as input to compute the nodes of an FTS tree, where **H** is used to compute the interior nodes and **F** is used to compute the leaf nodes (see Algorithm 15). For an FORC tree hash address, **layerAddr** = 0, **treeAddr** $\in \{0, \dots, 2^{h-h_0} - 1\}$, and the value of **keyPairAddr** indicates the index of the leaf of an XMSS tree with which the FTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_0} - 1\}$. The value of **treeHeight** indicates the height of the node being computed within an FORC tree, whose

range is $\{0, \dots, a\}$. Note that the height of the root node is a and the nodes with height 0 are leaf nodes. Let $\text{treeHeight} = z$. The value of treeIndex indicates the position of the node being computed within a specific level, whose range is $\{0, \dots, k \cdot 2^{a-z} - 1\}$.

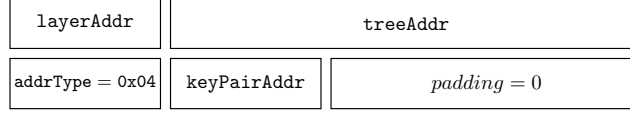


Figure 1.7: FTS public key compression address

As shown in Figure 1.7, $\text{addrType} = \text{FTS_ROOT} = 0x04$ for an FORC public key compression address. $\mathbf{T}_k$ takes an FORC public key compression address as input to compute the root of an FORC instance (see Line 26 of Algorithm 18). For an FORC public key compression address, $\text{layerAddr} = 0$, $\text{treeAddr} \in \{0, \dots, 2^{h-h_0} - 1\}$, and $\text{treeHeight} = 0$. The value of keyPairAddr indicates the index of the leaf of an XMSS tree with which the FTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_0} - 1\}$.

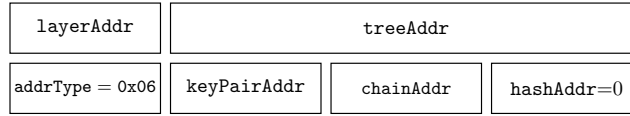


Figure 1.8: FORC secret key generation address

As shown in Figure 1.8, $\text{addrType} = \text{FTS_PRF} = 0x06$ for an FORC secret key generation address. $\mathbf{PRF}$ takes an FORC secret key generation address as input to compute the secret values of an FORC instance (see Line 6 of Algorithm 13). For an FORC secret key generation address, $\text{layerAddr} = 0$, $\text{treeAddr} \in \{0, \dots, 2^{h-h_0} - 1\}$, and $\text{hashAddr} = 0$. The value of keyPairAddr indicates the index of the leaf of an XMSS tree with which the FTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_0} - 1\}$. The value of chainAddr indicates the index of the secret value being computed, whose range is $\{0, \dots, k \cdot 2^a - 1\}$.

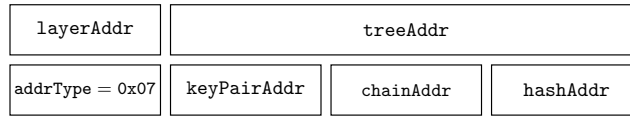


Figure 1.9: FORC hash chain address

As shown in Figure 1.9, $\text{addrType} = \text{FTS_CHAIN} = 0x07$ for an FORC hash chain address. An FORC hash chain address is used as input to $\mathbf{F}$ for computing the hash chains of FORC signatures (see Line 7 of Algorithm 15). For an FORC hash chain address, $\text{layerAddr} = 0$, $\text{treeAddr} \in \{0, \dots, 2^{h-h_0} - 1\}$. The value of keyPairAddr indicates the index of the leaf of an XMSS tree with which the FTS instance concerned is associated, whose range is $\{0, 1, \dots, 2^{h_0} - 1\}$. The value of chainAddr indicates the index of the chain being computed, whose range is $\{0, \dots, k \cdot 2^a - 1\}$. The value of hashAddr indicates the relative position of a node being computed within a specific hash chain, whose range is $\{1, \dots, w' - 1\}$.

1.6 The One-time Signature (OTS) Scheme

This section presents the inhomogeneous WOTS^α one-time signature scheme, which is an inhomogeneous variant [2] of WOTS^α [3]. Inhomogeneous WOTS^α uses three parameters, including n , len , and $\mathbf{w} = (w_0, \dots, w_{\text{len}-1})$. The security parameter n is the length in bytes of the messages that may be signed as well as the length of the private key elements, public key elements, and signature elements. The 2nd parameter len indicates the number of hash chains as well as the number of n -byte private key elements in the inhomogeneous WOTS^α . $\mathbf{w} = (w_0, \dots, w_{\text{len}-1})$ is the generalized Winternitz parameter, indicating that there are w_j nodes in the j -th hash chain.

An inhomogeneous WOTS^α private key consists of len secret n -byte values. In CEDRUS^α , these are all generated from an n -byte secret seed $\mathbf{SK.seed}$ using $\mathbf{PRF}$. Hash chains of different lengths are then created from the secret values using a chaining function, and the end nodes from each of the chains are hashed to derive the inhomogeneous WOTS^α public key. The hash chains are computed by the `chain` function given by Algorithm 4. The `chain` function takes an n -byte string $\mathbf{n}$ and integers s and i (where $i + s < w$) as input and returns the result of iterating a hash function $\mathbf{F}$ on the input s times, starting from an index of i . The `chain` function also requires as input $\mathbf{PK.seed}$, which is part of the CEDRUS^α public key, and an address ADRS . The type in ADRS must be set to `OTS_HASH`, and the layer address, tree address, key pair address, and chain address must be set to the address of the chain being computed. The `chain` function updates the hash address in ADRS with each iteration to specify the current position in the chain prior to $\mathbf{F}$'s use in $\mathbf{F}$.

Algorithm 4: `chain( $\mathbf{n}, i, s, \mathbf{PK.seed}, \text{ADRS}$ )`

Input: An n -byte element $\mathbf{n}$, starting index i , number of steps s , the public seed $\mathbf{PK.seed}$, and the address ADRS
Output: An n -byte element (a node of the hash chain)

```

1 ASSERT  $\text{ADRS.addrType} = \text{OTS\_Hash}$ 
2  $\text{tmp} \leftarrow \mathbf{n}$ 
3 for  $j$  from  $i$  to  $i + s - 1$  do
4    $\text{ADRS.setHashAddress}(j)$ 
5    $\text{tmp} \leftarrow \mathbf{F}(\mathbf{PK.seed}, \text{ADRS}, \text{tmp})$ 
6 return  $\text{tmp}$ 

```

1.6.1 OTS Public Key Generation

The $\text{pkGen}_{\text{OTS}}$ function given by Algorithm 5 generates the WOTS^α public keys. It takes $\mathbf{SK.seed}$ and $\mathbf{PK.seed}$ from the WOTS^α private key and an address as input. The type in the address ADRS must be set to `OTS_HASH`, and the layer address, tree address, and key pair address must encode the address of the WOTS^α public key to be generated. Lines 5 through 9 in Algorithm 5 generate the end nodes of the hash chains. For each of the len end nodes, the corresponding secret value is generated in line 7, and the `chain` function is called to compute the end node of the hash chain. Once the len end nodes are computed, they are compressed into a single n -byte value in lines 10 through 13.

Algorithm 5: $\text{pkGen}_{\text{OTS}}(\mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{ADRS})$

Input: Secret seed $\mathbf{SK}.\text{seed}$, public seed $\mathbf{PK}.\text{seed}$, the address ADRS
Output: The n -byte public key of the OTS (a leaf node of an XMSS tree)

```

1 ASSERT  $\text{ADRS}.\text{addrType} = \text{OTS\_Hash}$ 
2  $\text{skADRS} \leftarrow \text{ADRS}$ 
3  $\text{skADRS}.\text{setTypeAndClear}(\text{OTS\_PRF})$ 
4  $\text{skADRS}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
5 for  $i$  from 0 to  $\text{len} - 1$  do
6    $\text{skADRS}.\text{setChainAddress}(i)$ 
7    $sk \leftarrow \text{PRF}(\mathbf{PK}.\text{seed}, \mathbf{SK}.\text{seed}, \text{skADRS})$ 
8    $\text{ADRS}.\text{setChainAddress}(i)$ 
9    $\text{tmp}[i] \leftarrow \text{chain}(sk, 0, w_i - 1, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
10  $\text{otspkADRS} \leftarrow \text{ADRS}$ 
11  $\text{otspkADRS}.\text{setTypeAndClear}(\text{OTS\_PK})$ 
12  $\text{otspkADRS}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
13  $pk \leftarrow \mathbf{T}_{\text{len}}(\mathbf{PK}.\text{seed}, \text{otspkADRS}, \text{tmp})$ 
14 return  $pk$ 
```

One node in chain 0	(n bytes)
...	...
One node in chain $\text{len} - 1$	(n bytes)

Figure 1.10: Inhomogeneous WOTS^α signature data format

1.6.2 OTS Signature Generation

An inhomogeneous WOTS^α signature is a $\text{len} \cdot n$ -byte string, containing an array of len n -byte strings as shown in Figure 1.10. The Sign_{OTS} function given by Algorithm 6 generates the inhomogeneous WOTS^α signature. In addition to the n -byte message to be signed, Sign_{OTS} takes $\mathbf{SK}.\text{seed}$ and $\mathbf{PK}.\text{seed}$ from the CEDRUS^α private key and an address as input. The type in the address ADRS must be set to OTS_HASH , and the layer address, tree address, and key pair address must encode the address of the WOTS^α key that is used to sign the message. Sign_{OTS} encodes the n -byte message $\mathbf{n}$ into a vector $(s_0, \dots, s_{\text{len}-1})$ of len integers in $[w_0] \times \dots \times [w_{\text{len}-1}]$ (Line 5). For each of the len integers, Lines 10 through 13 compute the node in the hash chain that corresponds to the integer. The inhomogeneous WOTS^α signature contains the len computed n -byte nodes.

1.6.3 Computing an OTS Public Key From a Signature

Verifying a WOTS^α signature involves computing a public-key value from a message and signature value. Verification succeeds if the correct public-key value is computed, which is determined by using the computed public-key value along with other information to compute a candidate $\mathbf{PK}.\text{root}$ value and then comparing that value to the known value of $\mathbf{PK}.\text{root}$ from the CEDRUS^α public key. This section describes $\text{pkFromSig}_{\text{OTS}}$ (Algorithm 7), a function

Algorithm 6: $\text{Sig}_{\text{OTS}}(\mathbf{n}, \mathbf{SK.seed}, \mathbf{PK.seed}, \text{ADRS})$

Input: An n -byte element $\mathbf{n}$ (typically the root of an XMSS tree or PORs tree),
 Secret seed $\mathbf{SK.seed}$, public seed $\mathbf{PK.seed}$, and the address ADRS

Output: OTS signature

```

1 rootHashAddr  $\leftarrow$  ADRS
2 rootHashAddr.setTypeAndClear(ROOT_HASH)
3 rootHashAddr.setKeyPairAddress(ADRS.getKeyPairAddress())
4  $\text{cs} \leftarrow \left\lfloor \frac{(w_0-1)+\dots+(w_{\text{len}-1}-1)}{2} \right\rfloor$ 
5  $(s_0, \dots, s_{\text{len}-1}) \leftarrow \rho_{\mathbf{w}}^{\text{cs}}(\text{Int}(\mathbf{n}))$ 
6 skADRS  $\leftarrow$  ADRS
7 skADRS.setTypeAndClear(OTS_PRF)
8 skADRS.setKeyPairAddress(ADRS.getKeyPairAddress())
9 for for  $i$  from 0 to  $\text{len} - 1$  do
10   skADRS.setChainAddress( $i$ )
11    $sk \leftarrow \text{PRF}(\mathbf{PK.seed}, \mathbf{SK.seed}, \text{skADRS})$ 
12   ADRS.setChainAddress( $i$ )
13    $\text{sig}[i] \leftarrow \text{chain}(sk, 0, s_i, \mathbf{PK.seed}, \text{ADRS})$ 
14 return sig
```

that computes a candidate WOTS^α public key from a WOTS^α signature and corresponding message.

In addition to an n -byte message M and a $\text{len} \cdot n$ -byte signature $\mathbf{SIG}_{\text{OTS}}$, the $\text{pkFromSig}_{\text{OTS}}$ function takes $\mathbf{PK.seed}$ from the CEDRUS^α public key and an address as input. The type of the address ADRS must be set to OTS_HASH , and the layer address, tree address, and key pair address must encode the address of the WOTS^α key that was used to sign the message.

Lines ?? and 1 of Algorithm 7 extract the counter and array of len n -byte hash chain nodes. Lines 6 through 8 compute the end nodes for each of the chains using the signature value as the starting point and the message value to determine the number of iterations that need to be performed to get to the end node. Finally, the computed end nodes are compressed by Line 12.

1.7 eXtended Merkle Signature Scheme (XMSS)

An XMSS tree in layer i has 2^{h_i} leaves with each one being the public key of an inhomogeneous WOTS^α one-time signature. It allows the leaf nodes to be authenticated using a single n -byte XMSS public key (i.e., the root of the XMSS tree). An XMSS signature is $\text{len} \cdot n + h_i \cdot n$ bytes in length and consists of an inhomogeneous WOTS^α signature and an authentication path as illustrated in Figure 1.11. The authentication path is an array of nodes from the Merkle tree – one from each level of the tree, except for the root – that allows the verifier to compute the root of the XMSS tree when used in conjunction with the public key of the one-time signature.

Algorithm 7: $\text{pkFromSig}_{\text{OTS}}(\text{SIG}_{\text{OTS}}, M, \text{PK.seed}, \text{ADRS})$

Input: OTS signature SIG_{OTS} , message M , public seed PK.seed , and the address ADRS

Output: The OTS public key derived from SIG_{OTS}

```

1  $\text{sig} \leftarrow \text{SIG}_{\text{OTS}}.\text{getNodeArray}()$ 
2  $\text{rootHashAddr} \leftarrow \text{ADRS}$ 
3  $\text{rootHashAddr}.\text{setTypeAndClear}(\text{ROOT\_HASH})$ 
4  $\text{rootHashAddr}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
5  $(s_0, \dots, s_{\text{len}-1}) \leftarrow \rho_{\mathbf{w}}^{\text{cs}}(\text{Int}(M))$ 
6 for  $i$  from 0 to  $\text{len} - 1$  do
7    $\text{ADRS}.\text{setChainAddress}(i)$ 
8    $\text{tmp}[i] \leftarrow \text{chain}(\text{sig}[i], s_i, w - 1 - s_i, \text{PK.seed}, \text{ADRS})$ 
9  $\text{otspkADRS} \leftarrow \text{ADRS}$ 
10  $\text{otspkADRS}.\text{setTypeAndClear}(\text{OTS\_PK})$ 
11  $\text{otspkADRS}.\text{setKeyPairAddress}(\text{ADRS}.\text{getKeyPairAddress}())$ 
12  $\text{pk}_{\text{OTS}} \leftarrow \text{T}_{\text{len}}(\text{PK.seed}, \text{otspkADRS}, \text{tmp})$ 
13 return  $\text{pk}_{\text{OTS}}$ 
```

SIG_{OTS} ($\text{len} \cdot n$ bytes)
The 0-th element of AUTH (n bytes)
...
The $(h_i - 1)$ -th element of AUTH (n bytes)

Figure 1.11: XMSS signature data format for the i -th layer

1.7.1 Generating a Merkle Hash Tree

The xmssNode function (Algorithm 8) computes the nodes of an XMSS tree. The xmssNode function takes as input SK.seed and PK.seed from the private key, a target node index i (the index of the node being computed), a target node height z (the height within the Merkle tree of the node being computed) and an address. The address ADRS must have the layer address and tree address set to the XMSS tree within which the node is being computed.

Each node in an XMSS tree is the root of a subtree, and Algorithm 8 computes the root of the subtree recursively. If the subtree consists of a single leaf node, then the function simply returns the value of the corresponding WOTS^α public key (Lines 2 through 4). Otherwise, the function computes the roots of the left subtree (Line 6) and right subtree (Line 7) and hashes them together (Line 11).

1.7.2 Generating an XMSS Signature

The $\text{Sign}_{\text{XMSS}}$ function (Algorithm 9) creates an XMSS signature on an n -byte message M by creating an authentication path (Lines 3 through 4) and signing M with the appropriate WOTS^α key (Lines 5 through 7). In addition to M , $\text{Sign}_{\text{XMSS}}$ takes SK.seed and PK.seed from the CEDRUS $^\alpha$ private key, an address, and an index as input. The address ADRS must

Algorithm 8: $\text{xmssNode}(\mathbf{SK.seed}, i, z, \mathbf{PK.seed}, \text{ADRS})$

Input: Secret $\mathbf{SK.seed}$, target node index i , target node height z , public seed $\mathbf{PK.seed}$, address ADRS
Output: n -byte XMSS tree node

```

1 if  $z = 0$  then
2    $\text{ADRS.setTypeAndClear}(\text{OTS\_HASH})$ 
3    $\text{ADRS.setKeyPairAddress}(i)$ 
4    $node \leftarrow \text{pkGen}_{\text{OTS}}(\mathbf{SK.seed}, \mathbf{PK.seed}, \text{ADRS})$ 
5 else
6    $lnode \leftarrow \text{xmssNode}(\mathbf{SK.seed}, 2i, z - 1, \mathbf{PK.seed}, \text{ADRS})$ 
7    $rnnode \leftarrow \text{xmssNode}(\mathbf{SK.seed}, 2i + 1, z - 1, \mathbf{PK.seed}, \text{ADRS})$ 
8    $\text{ADRS.setTypeAndClear}(\text{XMSS\_TREE})$ 
9    $\text{ADRS.setTreeHeight}(z)$ 
10   $\text{ADRS.setTreeIndex}(i)$ 
11   $node \leftarrow \mathbf{H}(\mathbf{PK.seed}, \text{ADRS}, lnode \parallel rnnode)$ 
12 return  $node$ 

```

have the layer address and tree address set to the XMSS key that is being used to sign the message, and the index idx must be the index of the WOTS^α key within the XMSS tree that will be used to sign the message. The authentication path consists of the sibling nodes of each node that is on the path from the WOTS^α key used to the root. In Line 3 of Algorithm 9, $\lfloor \text{idx}/2^j \rfloor$ is the on path node, and $\lfloor \text{idx}/2^j \rfloor \oplus 1$ is the authentication path node. Line 4 computes the value of the authentication path node.

Algorithm 9: $\text{Sign}_{\text{XMSS}}(M, \mathbf{SK.seed}, \text{idx}, \mathbf{PK.seed}, \text{ADRS})$

Input: n -byte message M , secret seed $\mathbf{SK.seed}$, index idx , public seed $\mathbf{PK.seed}$, address ADRS
Output: XMSS signature σ_{XMSS}

```

1  $i \leftarrow \text{ADRS.getLayerAddress}()$ 
2 for  $j$  from 0 to  $h_i - 1$  do
3    $k \leftarrow \lfloor \text{idx}/2^j \rfloor \oplus 1$ 
4    $\text{Auth}[j] \leftarrow \text{xmssNode}(\mathbf{SK.seed}, k, j, \mathbf{PK.seed}, \text{ADRS})$ 
5  $\text{ADRS.setTypeAndClear}(\text{OTS\_HASH})$ 
6  $\text{ADRS.setKeyPairAddress}(\text{idx})$ 
7  $\sigma_{\text{OTS}} \leftarrow \text{Sign}_{\text{OTS}}(M, \mathbf{SK.seed}, \mathbf{PK.seed}, \text{ADRS})$ 
8  $\sigma_{\text{XMSS}} \leftarrow \sigma_{\text{OTS}} \parallel \text{Auth}$ 
9 return  $\sigma_{\text{XMSS}}$ 

```

1.7.3 Computing an XMSS Public Key from a XMSS Signature

Verifying an XMSS signature from the i -th layer involves computing a public-key value from a message and an XMSS signature value. Verification succeeds if the correct public-key value is computed, which is determined by using the computed public-key value along with other

information to compute a candidate **PK.root** value and then comparing that value to the known value of **PK.root** from the CEDRUS^α public key. This section describes **pkFromSig_{XMSS}** (Algorithm 10), a function that computes a candidate XMSS public key from an XMSS signature and corresponding message.

In addition to an n -byte message M and an $(\text{len} + h_i) \cdot n$ -byte signature **SIG_{XMSS}**, **pkFromSig_{XMSS}** takes **PK.seed** from the CEDRUS^α public key, an address, and an index as input. The address **ADRS** must be set to the layer address and tree address of the XMSS key that was used to sign the message, and the index **idx** must be the index of the WOTS^α key within the XMSS tree that was used to sign the message. Algorithm 10 begins by computing the WOTS^α public key in Lines 1 through 5. The root is then computed in Lines 6 through 17. Starting with the leaf node (i.e., the WOTS^α public key), a node at each level of the tree is computed by hashing together the node computed in the previous iteration with the corresponding authentication path node. In Lines 13 and 16, **AUTH** is interpreted as an array of h_i n -byte strings.

Algorithm 10: **pkFromSig_{XMSS}(idx, SIG_{XMSS}, M, PK.seed, ADRS)**

Input: Index **idxLeaf**, XMSS signature **SIG_{XMSS}**, n -byte message M , public seed **PK.seed**, and address **ADRS**
Output: n -byte root value of an XMSS tree

```

1 ADRS.setTypeAndClear(OTS_HASH)
2 ADRS.setKeyPairAddress(idx)
3 SIG ← SIGXMSS.getSigOTS()
4 AUTH ← SIG.getAuthXMSS()
5 node[0] ← pkFromSigOTS(SIG, M, PK.seed, ADRS)
6 ADRS.setTypeAndClear(XMSS_TREE)
7 ADRS.setTreeIndex(idx)
8 i ← ADRS.getLayerAddress()
9 for k from 0 to  $h_i - 1$  do
10   ADRS.setTreeHeight( $k + 1$ )
11   if  $\lfloor \text{idx}/2^k \rfloor$  is even then
12     ADRS.setTreeIndex(ADRS.getTreeIndex()/2)
13     node[1] ← H(PK.seed, ADRS, node[0] || AUTH[k])
14   else
15     ADRS.setTreeIndex((ADRS.getTreeIndex() - 1)/2)
16     node[1] ← H(PK.seed, ADRS, AUTH[k] || node[0])
17   node[0] ← node[1]
18 return node[0]
```

1.8 The CEDRUS^α Hypertree Signature

As described in Section 1.4, a CEDRUS^α hypertree is a tree of XMSS trees. The XMSS keys at the lowest layer are used to sign PORs public keys, and the XMSS keys at every other layer are used to sign the XMSS public keys at the layer below. The top layer (layer $d - 1$) is a single XMSS tree, and the public key of this XMSS key pair (i.e., the root of the Merkle tree) is the public key of the hypertree (**PK.root**). The lowest layer has 2^{h-h_0} XMSS trees, providing 2^h WOTS^α keys to sign PORs keys.

1.8.1 Hypertree Signature Generation

A CEDRUS^α hypertree signature is $(d \cdot \text{len} + h) \cdot n$ bytes in length and consists of a sequence of d XMSS signatures, starting with a signature generated using an XMSS key at the lowest layer and ending with a signature generated using the XMSS key at the top layer (see Figure 1.12). In addition to the n -byte message M , the sign_{HT} function given by Algorithm 11 takes as input **SK.seed** and **PK.seed** from the CEDRUS^α private key, the index **idxTree** of the XMSS tree at the lowest layer that will sign the message, and the index **idxLeaf** of the WOTS^α key within the XMSS tree that will sign the message.

SIG _{XMSS} for layer 0 $((h_0 + \text{len}) \cdot n \text{ bytes})$
SIG _{XMSS} for layer 1 $((h_1 + \text{len}) \cdot n \text{ bytes})$
... ..
SIG _{XMSS} for layer $d - 1$ $((h_{d-1} + \text{len}) \cdot n \text{ bytes})$

Figure 1.12: Hypertree signature data format

Algorithm 11 begins in Lines 1 through 3 by signing M with the specified XMSS key using the WOTS^α key within that XMSS key specified by **idxLeaf**. The XMSS public key is obtained (Line 5 or 14) for each successive layer and signed by the appropriate key at the next higher level (Lines 7 through 11).

1.8.2 Hypertree Signature Verification

Hypertree signature verification works by making d calls to **pkFromSig**_{XMSS} (Algorithm 10) and comparing the result to the public key of the hypertree. In addition to the n -byte message M and the $(d \cdot \text{len} + h) \cdot n$ -byte signature **SIG**_{HT}, **verify**_{HT} (Algorithm 12) takes as input **PK.seed** and **PK.root** from the CEDRUS^α public key, the index **idxTree** of the XMSS tree at the lowest layer that signed the message, and the index **idxLeaf** of the WOTS^α key within the XMSS tree that signed the message. At each layer, either the message M or the computed public key of the XMSS key at the lower layer is provided along with the appropriate XMSS signature to **pkFromSig**_{XMSS} in order to obtain the layer's computed XMSS public key. If the computed XMSS public key of the top layer tree is the same as the known hypertree public key, **PK.root**, then verification succeeds.

1.9 The FORC Few-time Signature

FORC is a few-time signature scheme that is used to sign the digests of the actual messages. Unlike one-time signatures, for which forgeries become feasible if a key is used twice, the

Algorithm 11: $\text{sign}_{\text{HT}}(M, \mathbf{SK}.\text{seed}, \mathbf{PK}.\text{seed}, \text{idxTree}, \text{idxLeaf})$

Input: Message $M \in \mathbb{F}_2^n$, private seed $\mathbf{SK}.\text{seed}$, public seed $\mathbf{PK}.\text{seed}$, XMSS tree index idxTree , leaf index idxLeaf

Output: Hypertree signature σ_{HT}

```

1  ADRS  $\leftarrow$  toByte(0, 32)
2  ADRS.setTreeAddress(idxTree)
3   $\sigma_{\text{XMSS}} \leftarrow \text{sign}_{\text{XMSS}}(M, \mathbf{SK}.\text{seed}, \text{idxLeaf}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
4   $\sigma_{\text{HT}} \leftarrow \sigma_{\text{XMSS}}$ 
5   $root \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \sigma_{\text{XMSS}}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
6  for  $j$  from 1 to  $d - 1$  do
7     $\text{idxLeaf} \leftarrow \text{idxTree} \bmod 2^{h_j}$ 
8     $\text{idxTree} \leftarrow \text{idxTree} \gg h_j$ 
9    ADRS.setLayerAddress( $j$ )
10   ADRS.setTreeAddress(idxTree)
11    $\sigma_{\text{XMSS}} \leftarrow \text{Sign}_{\text{XMSS}}(root, \mathbf{SK}.\text{seed}, \text{idxLeaf}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
12    $\sigma_{\text{HT}} \leftarrow \sigma_{\text{HT}} \parallel \sigma_{\text{XMSS}}$ 
13   if  $j < d - 1$  then
14      $root \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \sigma_{\text{XMSS}}, root, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
15 return  $\sigma_{\text{HT}}$ 

```

Algorithm 12: $\text{verify}_{\text{HT}}(M, \mathbf{SIG}_{\text{HT}}, \mathbf{PK}.\text{seed}, \text{idxTree}, \text{idxLeaf}, \mathbf{PK}.\text{root})$

Input: Message $M \in \mathbb{F}_2^n$, signature $\mathbf{SIG}_{\text{HT}}$, public seed $\mathbf{PK}.\text{seed}$, XMSS tree index idxTree , leaf index idxLeaf , public key $\mathbf{PK}.\text{root}$

Output: Boolean

```

1  ADRS  $\leftarrow$  toByte(0, 32)
2  ADRS.setTreeAddress(idxTree)
3   $\mathbf{SIG}_{\text{tmp}} \leftarrow \mathbf{SIG}_{\text{HT}}.\text{getXMSSSignature}(0)$ 
4   $node \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \mathbf{SIG}_{\text{tmp}}, M, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
5  for  $j$  from 1 to  $d - 1$  do
6     $\text{idxLeaf} \leftarrow \text{idxTree} \bmod 2^{h_j}$ 
7     $\text{idxTree} \leftarrow \text{idxTree} \gg h_j$ 
8    ADRS.setLayerAddress( $j$ )
9    ADRS.setTreeAddress(idxTree)
10    $\mathbf{SIG}_{\text{tmp}} \leftarrow \mathbf{SIG}_{\text{HT}}.\text{getXMSSSignature}(j)$ 
11    $node \leftarrow \text{pkFromSig}_{\text{XMSS}}(\text{idxLeaf}, \mathbf{SIG}_{\text{tmp}}, node, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
12 if  $node = \mathbf{PK}.\text{root}$  then
13   return true
14 else
15   return false

```

security of a FORC key degrades gradually as the number of signatures increases. FORC uses three parameters: k , a and w' . A FORC private key consists of k sets of $t = 2^a$ n -byte

strings, all of which are pseudorandomly generated from the seed **SK.seed**. Each private key string is used as the first node of a hash chain of length w' , and the last node of the chain are used as the leaf of the corresponding Merkle tree. The roots of the k trees are hashed together to form the FORC public key. A signature on the message digest consists of k selected hash-chain nodes, along with the authentication paths for the corresponding leaves (see Figure 1.13).

Private key element for tree 0 (n bytes)
AUTH for tree 0 ($a \cdot n$ bytes)
...
Private key element for tree $k - 1$ (n bytes)
AUTH for tree 0 ($a \cdot n$ bytes)

Figure 1.13: FORC signature data format

1.9.1 Generating FORC Secret Values

The `skGenFORC` function (Algorithm 13) generates the n -byte strings of the FORC private key. The function takes **SK.seed** and **PK.seed** from the CEDRUS^α private key, an address, and an index as input. The type in the address `ADRS` must be set to `FTS_PRF`, and the tree address and key pair address must be set to the index of the inhomogeneous WOTS^α key within the XMSS tree that signs the FORC key. The hash address must be set to zero. The chain address `idx` is the index of the FORC secret value, or equivalently the index of the hash chain whose first node is being generated.

Algorithm 13: `skGenFORC(SK.seed, PK.seed, ADRS, idx)`

Input: Secret seed **SK.seed**, public seed **PK.seed**, address `ADRS`, chain index `idx`
Output: n -byte FORC private key value

```

1 skADRS ← ADRS
2 skADRS.setTypeAndClear(FTS_PRF)
3 skADRS.setKeyPairAddress(ADRS.getKeyPairAddress())
4 skADRS.setChainAddress(idx)
5 skADRS.setHashAddress(0)
6 return PRF(PK.seed, SK.seed, skADRS)
```

1.9.2 Computing the Nodes of a FORC Tree

The `chainFORC` function (Algorithm 14) computes a node in a FORC hash chain. The function takes an n -byte chain node, the index of this node in the chain, the number of hash steps to compute, **PK.seed**, and an address as input. The type in the address `ADRS` must be set to `FTS_CHAIN`, and the chain address must be set to the index of the chain being computed.

The `nodeFORC` function (Algorithm 15) computes the nodes of a FORC tree. It is the same as `xmssNode`, except that the leaf nodes are the last nodes of the FORC hash chains instead

Algorithm 14: chainFORC($n, i, s, \mathbf{PK.seed}, \text{ADRS}$)

Input: n -byte chain node n , start index i , number of steps s , public seed $\mathbf{PK.seed}$, address ADRS

Output: An n -byte node of the FORC hash chain

```

1 tmp  $\leftarrow n$ 
2 for  $j$  from  $i + 1$  to  $i + s$  do
3   ADRS.setHashAddress( $j$ )
4   tmp  $\leftarrow F(\mathbf{PK.seed}, \text{ADRS}, \text{tmp})$ 
5 return tmp

```

of inhomogeneous WOTS^α public keys. The nodeFORC function takes as input $\mathbf{SK.seed}$ and $\mathbf{PK.seed}$ from the CEDRUS^α private key, a target node index i (the index of the node being computed), a target node height z (the height within the FORC tree of the node being computed), and an address. The address ADRS must have the layer address set to zero (since the XMSS tree that signs a FORC key is always at layer 0), the tree address set to the XMSS tree that signs the FORC key, the type set to FTS.TREE , and the key pair address set to the index of the inhomogeneous WOTS^α key within the XMSS tree that signs the FORC key. The target node height and index must satisfy $z \leq a$ and $i < k \cdot 2^{a-z}$.

Each node in the FORC tree is the root of a subtree, and Algorithm 15 computes the root of a subtree recursively. If the subtree consists of a single leaf node, then the function generates the first node of the corresponding hash chain and computes the last node of this chain (Lines 1 through 7). Otherwise, the function computes the roots of the left subtree (Line 12) and right subtree (Line 13) and hashes them together (Line 16).

1.9.3 Generating a FORC Signature

The signFORC function (Algorithm 17) signs a $k \cdot (a + \log_2 w')$ -bit message digest md . In addition to the message digest, signFORC takes $\mathbf{SK.seed}$ and $\mathbf{PK.seed}$ from the CEDRUS^α private key and an address as input. The address ADRS must have the layer address set to zero (since the XMSS tree that signs a FORC key is always at layer 0), the tree address set to the XMSS tree that signs the FORC key, the type set to FTS.TREE , and the key pair address set to the index of the inhomogeneous WOTS^α key within the XMSS tree that signs the FORC key.

The signFORC function splits md into k a -bit strings and $k \log_2 w'$ -bit strings (Line 2). The a -bit strings are interpreted as integers between 0 and $t - 1$ and select one hash chain from each of the k sets. The $\log_2 w'$ -bit strings are interpreted as integers between 0 and $w' - 1$ and select one node on each selected hash chain. This parsing is computed by msgToIndices (Algorithm 16). In Algorithm 16, $\text{md}[\text{offset}]$ denotes the offset -th bit of md . For each selected hash-chain node, an authentication path for the corresponding leaf is computed and added to the signature (Lines 11 through 14).

1.9.4 Computing a FORC Public Key from a Signature

This section describes pkFromSigFORC (Algorithm 18), a function that computes a candidate FORC public key from a FORC signature and corresponding message digest. In addition to a message digest md and a $k \cdot (a + 1) \cdot n$ -byte signature $\mathbf{SIG}_{\text{FORC}}$, pkFromSigFORC takes

Algorithm 15: nodeFORC(**SK.seed**, i, z , **PK.seed**, ADRS)

Input: Secret seed **SK.seed**, target node index i , target node height z , public seed **PK.seed**, address ADRS

Output: An n -byte FORC tree node

```

1 if  $z = 0$  then
2    $sk \leftarrow \text{skGenFORC}(\mathbf{SK.seed}, \mathbf{PK.seed}, \text{ADRS}, i)$ 
3    $\text{chainADRS} \leftarrow \text{ADRS}$ 
4    $\text{chainADRS.setTypeAndClear}(\text{FTS\_CHAIN})$ 
5    $\text{chainADRS.setKeyPairAddress}(\text{ADRS.getKeyPairAddress}())$ 
6    $\text{chainADRS.setChainAddress}(i)$ 
7    $\text{node} \leftarrow \text{chainFORC}(sk, 0, w', \mathbf{PK.seed}, \text{chainADRS})$ 
8 else
9    $\text{treeADRS} \leftarrow \text{ADRS}$ 
10   $\text{treeADRS.setTypeAndClear}(\text{FTS\_TREE})$ 
11   $\text{treeADRS.setKeyPairAddress}(\text{ADRS.getKeyPairAddress}())$ 
12   $\text{lnode} \leftarrow \text{nodeFORC}(\mathbf{SK.seed}, 2i, z - 1, \mathbf{PK.seed}, \text{treeADRS})$ 
13   $\text{rnode} \leftarrow \text{nodeFORC}(\mathbf{SK.seed}, 2i + 1, z - 1, \mathbf{PK.seed}, \text{treeADRS})$ 
14   $\text{treeADRS.setTreeHeight}(z)$ 
15   $\text{treeADRS.setTreeIndex}(i)$ 
16   $\text{node} \leftarrow \mathbf{H}(\mathbf{PK.seed}, \text{treeADRS}, \text{lnode} \parallel \text{rnode})$ 
17 return  $\text{node}$ 

```

Algorithm 16: msgToIndices(md, a, ℓ, k)

Input: Message digest md , tree height a , chain-index length ℓ , number of trees k

Output: Tree indices *indices* and chain indices *lengths*

```

1  $\text{offset} \leftarrow 0$ 
2 for  $i$  from 0 to  $k - 1$  do
3    $\text{indices}[i] \leftarrow 0$ 
4    $\text{lengths}[i] \leftarrow 0$ 
5   for  $j$  from 0 to  $a - 1$  do
6      $\text{indices}[i] \leftarrow \text{indices}[i] + \text{md}[\text{offset}] \cdot 2^j$ 
7      $\text{offset} \leftarrow \text{offset} + 1$ 
8   for  $j$  from 0 to  $\ell - 1$  do
9      $\text{lengths}[i] \leftarrow \text{lengths}[i] + \text{md}[\text{offset}] \cdot 2^j$ 
10     $\text{offset} \leftarrow \text{offset} + 1$ 
11 return ( $\text{indices}, \text{lengths}$ )

```

PK.seed from the CEDRUS ^{α} public key and an address as input. The address ADRS must have the layer address set to zero (since the XMSS tree that signs a FORC key is always at layer 0), the tree address set to the XMSS tree that signs the FORC key, the type set to FTS.TREE, and the key pair address set to the index of the WOTS ^{α} key within the XMSS tree that signs the FORC key.

The pkFromSigFORC function begins by computing the roots of each of the k Merkle trees

Algorithm 17: signFORC(md, **SK.seed**, **PK.seed**, ADRS)

Input: Message digest md, secret seed **SK.seed**, public seed **PK.seed**, address ADRS

Output: FORC signature σ_{FORC}

```

1  $\sigma_{\text{FORC}} \leftarrow \emptyset$ 
2  $(\text{indices}, \text{lengths}) \leftarrow \text{msgToIndices}(\text{md}, a, \log_2 w', k)$ 
3 for  $i$  from 0 to  $k - 1$  do
4    $\text{idx} \leftarrow i \cdot 2^a + \text{indices}[i]$ 
5    $sk \leftarrow \text{skGenFORC}(\text{SK.seed}, \text{PK.seed}, \text{ADRS}, \text{idx})$ 
6    $\text{chainADRS} \leftarrow \text{ADRS}$ 
7    $\text{chainADRS.setTypeAndClear}(\text{FTS\_CHAIN})$ 
8    $\text{chainADRS.setKeyPairAddress}(\text{ADRS.getKeyPairAddress}())$ 
9    $\text{chainADRS.setChainAddress}(\text{idx})$ 
10   $\sigma_{\text{FORC}} \leftarrow \sigma_{\text{FORC}} \parallel \text{chainFORC}(sk, 0, \text{lengths}[i], \text{PK.seed}, \text{chainADRS})$ 
11  for  $j$  from 0 to  $a - 1$  do
12     $s \leftarrow \lfloor \text{indices}[i] / 2^j \rfloor \oplus 1$ 
13     $\text{Auth}[j] \leftarrow \text{nodeFORC}(\text{SK.seed}, i \cdot 2^{a-j} + s, j, \text{PK.seed}, \text{ADRS})$ 
14   $\sigma_{\text{FORC}} \leftarrow \sigma_{\text{FORC}} \parallel \text{Auth}$ 
15 return  $\sigma_{\text{FORC}}$ 

```

(Lines 2 through 22). As in signFORC, the message digest is split into k a -bit strings and $k \log_2 w'$ -bit strings (Line 1). The a -bit strings determine the locations in the Merkle trees, and the $\log_2 w'$ -bit strings determine the locations on the corresponding hash chains. The chain nodes from the signature are advanced to the last of their chains (Line 11), and these leaf nodes are used along with the corresponding authentication paths from the signature to compute the Merkle tree roots (Lines 12 through 22). Once all of the Merkle tree roots have been computed, they are hashed together to compute the FORC public key (Lines 23 through 26).

1.10 The CEDRUS ^{α} Signature Scheme

The CEDRUS ^{α} private key contains a secret seed value and a secret PRF key. The public key consists of a key identifier **PK.seed** and the root of the hypertree. The parameters for CEDRUS ^{α} are those specified previously for WOTS ^{α} , XMSS, the CEDRUS ^{α} hypertree, and FORS. CEDRUS ^{α} uses one additional parameter m , which is the length in bytes of the message digest. It is computed as

$$m = \left\lceil \frac{k \cdot (a + \log_2 w')}{8} \right\rceil + \left\lceil \frac{h - h_0}{8} \right\rceil + \left\lceil \frac{h_0}{8} \right\rceil.$$

1.10.1 CEDRUS ^{α} Key Generation

CEDRUS ^{α} public keys contain two elements. The first is an n -byte public seed **PK.seed**, which is used in many hash function calls to provide domain separation between different CEDRUS ^{α} key pairs. The second value is the hypertree public key **PK.root** (i.e., the root of the top layer XMSS tree). **PK.seed** shall be generated by a random bit generator supporting

Algorithm 18: pkFromSigFORC(σ_{FORC} , md, **PK.seed**, ADRS)

Input: FORC signature σ_{FORC} , message digest md, public seed **PK.seed**, address ADRS

Output: FORC public key pk_{FORC}

```

1 (indices, lengths)  $\leftarrow$  msgToIndices(md, a,  $\log_2 w'$ , k)
2 for i from 0 to k - 1 do
3   idx  $\leftarrow i \cdot 2^a + \text{indices}[i]$ 
4   sigNode  $\leftarrow \sigma_{\text{FORC}}.\text{getNode}(i)$ 
5   ADRS.setTreeHeight(0)
6   ADRS.setTreeIndex(idx)
7   chainADRS  $\leftarrow$  ADRS
8   chainADRS.setTypeAndClear(FTS_CHAIN)
9   chainADRS.setKeyPairAddress(ADRS.getKeyPairAddress())
10  chainADRS.setChainAddress(idx)
11  node[0]  $\leftarrow$  chainFORC(sigNode, lengths[i],  $w' - \text{lengths}[i]$ , PK.seed, chainADRS)

12  Auth  $\leftarrow \sigma_{\text{FORC}}.\text{getAuth}(i)$ 
13  for j from 0 to a - 1 do
14    ADRS.setTreeHeight(j + 1)
15    if  $\lfloor \text{indices}[i] / 2^j \rfloor \bmod 2 = 0$  then
16      ADRS.setTreeIndex(ADRS.getTreeIndex() / 2)
17      node[1]  $\leftarrow \mathbf{H}(\mathbf{PK.seed}, \text{ADRS}, \text{node}[0] \parallel \text{Auth}[j])$ 
18    else
19      ADRS.setTreeIndex((ADRS.getTreeIndex() - 1) / 2)
20      node[1]  $\leftarrow \mathbf{H}(\mathbf{PK.seed}, \text{ADRS}, \text{Auth}[j] \parallel \text{node}[0])$ 
21    node[0]  $\leftarrow$  node[1]
22  root[i]  $\leftarrow$  node[0]

23 forspkADRS  $\leftarrow$  ADRS
24 forspkADRS.setTypeAndClear(FTS_ROOT)
25 forspkADRS.setKeyPairAddress(ADRS.getKeyPairAddress())
26  $pk_{\text{FORC}} \leftarrow \mathbf{T}_k(\mathbf{PK.seed}, \text{forspkADRS}, \text{root})$ 
27 return  $pk_{\text{FORC}}$ 

```

at least $8n$ bits of security strength. The CEDRUS^α private key contains two random, secret n -byte values, including **SK.seed** and **SK.prf**. **SK.seed** is used to generate all of the OTS and FTS key elements. **SK.prf** is used to generate a randomization value for the randomized hashing of the message in CEDRUS^α. The private key also includes a copy of the public key. Both **SK.seed** and **SK.prf** shall be generated by a random bit generator supporting at least $8n$ bits of security strength. Algorithm 19 generates a CEDRUS^α key pair. Lines 1 through 3 compute the root of the top layer XMSS tree. Line 4 bundles the three inputs and the computed **PK.root** into the private and public keys.

Algorithm 19: $\text{keyGen}_{\text{SLH}}(\mathbf{SK}.\text{seed}, \mathbf{SK}.\text{prf}, \mathbf{PK}.\text{seed})$ **Input:** None**Input:** Secret seed $\mathbf{SK}.\text{seed}$, PRF key $\mathbf{SK}.\text{prf}$, public seed $\mathbf{PK}.\text{seed}$ **Output:** CEDRUS^α key pair $(\mathbf{SK}, \mathbf{PK})$

```

1 ADRS  $\leftarrow$  toByte(0, 32)
2 ADRS.setLayerAddress( $d - 1$ )
3  $\mathbf{PK}.\text{root} \leftarrow \text{xmssNode}(\mathbf{SK}.\text{seed}, 0, h_{d-1}, \mathbf{PK}.\text{seed}, \text{ADRS})$ 
4 return  $((\mathbf{SK}.\text{seed}, \mathbf{SK}.\text{prf}, \mathbf{PK}.\text{seed}, \mathbf{PK}.\text{root}), (\mathbf{PK}.\text{seed}, \mathbf{PK}.\text{root}))$ 

```

$\mathbf{R}$ (n bytes)
$\mathbf{SIG}_{\text{FORS+C}}$ ($4 + k \cdot (a + 1) \cdot n$ bytes)
$\mathbf{SIG}_{\text{HT}}$ ($4 \cdot d + (h + d \cdot \text{len}) \cdot n$ bytes)

Figure 1.14: CEDRUS^α signature data format**1.10.2 CEDRUS^α Signature Generation**

A CEDRUS^α signature consists of an n -byte randomization string, a FORS signature, and a hypertree signature. Generating a CEDRUS^α signature (Algorithm 20) begins by creating an m -byte message digest (Lines 2 through 4). A PRF is used to create a message randomizer (Line 3), and it is hashed along with the message to create the digest (Line 4). Bits are then extracted from the message digest to be signed by the FORS key (Line 5), to select an XMSS tree (Lines 6 and 8), and to select a WOTS^α key and corresponding FORS key within that XMSS tree (Lines 7 and 9). Next, the FORS signature is computed (Lines 10 through 13), and the corresponding FORS public key is obtained (Line 14). Finally, the FORS public key is signed (Line 15).

The message randomizer may be set in either a non-deterministic or deterministic way, depending on whether `addrnd` is provided as an input. For the “hedged” variant, `addrnd` is provided as an input, and `optRand` is set to `addrnd`. The hedged variant is the default and should be used on platforms where side-channel attacks are a concern. When using the hedged version, `addrnd` shall be an n -byte random value. While `addrnd` should ideally be generated by a random bit generator. For the deterministic variant, `addrnd` is not provided as an input, and `optRand` is set to $\mathbf{PK}.\text{seed}$, which results in signing being deterministic (i.e., signing the same message twice will result in the same signature). The deterministic variant is available for platforms where a random bit generator is not available.

1.10.3 Signature Verification

As with signature generation, CEDRUS^α signature verification (Algorithm 21) begins by computing a message digest (Line 7) and then extracting `md` (Line 8), `idxTree` (Lines 9 and 11), and `idxLeaf` (Lines 10 and 12) from the digest. A candidate FORS public key is then computed (Line 16), and the signature on the FORS key is verified (Line 17). If this signature verification succeeds, then the correct FORS public key was computed, and the signature **SIG** on message `msg` is valid.

Algorithm 20: $\text{signInternal}_{\text{SLH}}(\text{msg}, \text{SK}, \text{addrnd})$

Input: Message msg , secret key $\text{SK} = (\text{SK.seed}, \text{SK.prf}, \text{PK.seed}, \text{PK.root})$,
and an n -byte random number addrnd (optional)

Output: SLH-DSA signature σ

- 1 $\text{ADRS} \leftarrow \text{toByte}(0, 32)$
- 2 $\text{optRand} \leftarrow \text{addrnd}$
- 3 $\mathbf{R} \leftarrow \text{PRF}_{\text{MSG}}(\text{SK.prf}, \text{optRand}, \text{msg})$
- 4 $\text{digest} \leftarrow \mathbf{H}_{\text{MSG}}(\mathbf{R}, \text{PK.seed}, \text{PK.root}, \text{msg})$
- 5 $\text{md} \leftarrow \text{digest}[0 : \lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil]$
- 6 $\text{idxTree} \leftarrow \text{digest}[\lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil : \lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil + \lceil \frac{h - h_0}{8} \rceil]$
- 7 $\text{idxLeaf} \leftarrow \text{digest}[\lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil + \lceil \frac{h - h_0}{8} \rceil : \lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil + \lceil \frac{h - h_0}{8} \rceil + \lceil \frac{h_0}{8} \rceil]$
- 8 $\text{idxTree} \leftarrow \text{toInt}(\text{idxTree}, \lceil (h - h_0)/8 \rceil) \bmod 2^{h - h_0}$
- 9 $\text{idxLeaf} \leftarrow \text{toInt}(\text{idxLeaf}, \lceil h_0/8 \rceil) \bmod 2^{h_0}$
- 10 $\text{ADRS.setTreeAddress}(\text{idxTree})$
- 11 $\text{ADRS.setTypeAndClear}(\text{FORS_TREE})$
- 12 $\text{ADRS.setKeyPairAddress}(\text{idxLeaf})$
- 13 $\sigma_{\text{FORS}} \leftarrow \text{sign}_{\text{FORS}}(\text{md}, \text{SK.seed}, \text{PK.seed}, \text{ADRS})$
- 14 $pk_{\text{FORS}} \leftarrow \text{pkFromSig}_{\text{FORS}}(\sigma_{\text{FORS}}, \text{md}, \text{PK.seed}, \text{ADRS})$
- 15 $\sigma_{\text{HT}} \leftarrow \text{sign}_{\text{HT}}(pk_{\text{FORS}}, \text{SK.seed}, \text{PK.seed}, \text{idxTree}, \text{idxLeaf})$
- 16 $\sigma \leftarrow \mathbf{R} \parallel \text{ctr}_{\text{FTS}} \parallel \sigma_{\text{FORS}} \parallel \sigma_{\text{HT}}$
- 17 **return** σ

1.11 Parameter Sets and Instantiations

We provide 8 parameter sets as shown in Table 1.1. Each parameter set name indicates the bit security against generic classical attacks, and whether the parameter set was designed to create relatively small signatures (**s**) or to have relatively fast signature generation (**f**). A parameter set consists of parameters for the one-time signature (n , **w**, and **len**), **XMSS** and the **CEDRUS^α** hypertree (h , **h** and d), the few-time signature (k , a and w') as well as instantiations for the cryptographic functions given in Table 1.2 and Table 1.3.

The functions described in Section 1.3 are instantiated differently in different algorithms as shown in Table 1.2 and Table 1.3. The functions **PRF**, **F**, **H**, and **T_l** are heavily employed in key generation, signing and verification. To increase their efficiency, we pad the **PK.seed** input to one message block (i.e., 64 bytes) when the functions are instantiated with **SM3** (see Table 1.2). Since **SM3** is designed based on the Merkle-Damgård mode, this approach facilitates the reuse of intermediate state value after one-block computation. Finally, ADRS^c in Table 1.2 represent the compression address of **ADRS**, where $\text{ADRS}^c.\text{layerAddr}$ is the lowest byte of **ADRS.layerAddr** and $\text{ADRS}^c.\text{addrType}$ is lowest byte of **ADRS.addrType**, $\text{ADRS}^c.\text{treeAddr}$ is the lowest 8 bytes of **ADRS.treeAddr**. Therefore, the size of ADRS^c is 22-byte, reducing the number of calls **SM3**.

Warning: In this document, the cryptographic functions described in Section 1.3 are instantiated with hash functions or eXtended Output Functions (XOF) derived from **SM3**,

Algorithm 21: $\text{verifyInternal}_{\text{SLH}}(\text{msg}, \text{SIG}, \text{PK})$ **Input:** Message msg , signature SIG , public key $\text{PK} = (\text{PK.seed}, \text{PK.root})$ **Output:** Boolean

```

1 if  $|\text{SIG}| \neq n + k \cdot (a + 1) \cdot n + d \cdot \text{len} \cdot n + h \cdot n$  then
2   return false
3  $\text{ADRS} \leftarrow \text{toByte}(0, 32)$ 
4  $\text{R} \leftarrow \text{SIG.getR}()$ 
5  $\text{SIG}_{\text{FORS}} \leftarrow \text{SIG.getSIG}_{\text{FORS}}()$ 
6  $\text{SIG}_{\text{HT}} \leftarrow \text{SIG.getSIG}_{\text{HT}}()$ 
7  $\text{digest} \leftarrow \text{H}_{\text{MSG}}(\text{R}, \text{PK.seed}, \text{PK.root}, \text{msg})$ 
8  $\text{md} \leftarrow \text{digest}[0 : \lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil]$ 
9  $\text{idxTree} \leftarrow \text{digest}[\lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil : \lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil + \lceil \frac{h - h_0}{8} \rceil]$ 
10  $\text{idxLeaf} \leftarrow \text{digest}[\lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil + \lceil \frac{h - h_0}{8} \rceil : \lceil \frac{k \cdot (a + \log_2 w')}{8} \rceil + \lceil \frac{h - h_0}{8} \rceil + \lceil \frac{h_0}{8} \rceil]$ 
11  $\text{idxTree} \leftarrow \text{toInt}(\text{idxTree}, \lceil (h - h_0)/8 \rceil) \bmod 2^{h - h_0}$ 
12  $\text{idxLeaf} \leftarrow \text{toInt}(\text{idxLeaf}, \lceil h_0/8 \rceil) \bmod 2^{h_0}$ 
13  $\text{ADRS.setTreeAddress}(\text{idxTree})$ 
14  $\text{ADRS.setTypeAndClear}(\text{PORS\_TREE})$ 
15  $\text{ADRS.setKeyPairAddress}(\text{idxLeaf})$ 
16  $pk_{\text{FORS}} \leftarrow \text{pkFromSig}_{\text{FORS}}(\text{SIG}_{\text{FORS}}, \text{md}, \text{PK.seed}, \text{ADRS})$ 
17 return  $\text{verify}_{\text{HT}}(pk_{\text{FORS}}, \text{SIG}_{\text{HT}}, \text{PK.seed}, \text{idxTree}, \text{idxLeaf}, \text{PK.root})$ 

```

which are provided in the `API_PKC.zip` file at https://niccs.org.cn/niccs/Download/API/API_PKC.zip. These functions are regarded as *ideal* primitives. For example, we assume that $\text{XOF}_{\text{SM3}}(\text{msg}, 8n)$ provides $8n$ -bit (classical) security against (2nd) preimage attacks for all $n \in \mathbb{Z}_+$ even though this is not the case in reality. In particular, we ignore the security flaws due to the Merkle-Damgård construction of SM3 [4]. These instantiations are made to conform the rules specified by NICCS and to facilitate performance comparisons among submitted algorithms. *The actual security strength of these instances may not reach the specified security level due to dedicated attacks on SM3 and its derived functions.* Eventually, the cryptographic functions described in Section 1.3 should be instantiated with secure hash functions and XOFs.

Table 1.1: CEDRUS^α parameter sets

Algorithm	n	h	d	a	k	$\mathbf{w}$	w'	$\mathbf{len}$	Classical Sec. (Bits)	Quantum Sec. (Bits)	SK Size (Bytes)	PK Size (Bytes)	Sig Size (Bytes)
CEDRUS ^α -160s	20	68	9	10	16	$[49]^2 \times [48]^{28}$	8	30	160	80	80	40	10300
CEDRUS ^α -160f	20	66	17	7	28	$[18]^{35} \times [17]^5$	4	40	160	80	80	40	19420
CEDRUS ^α -256s	32	67	9	12	23	$[46]^7 \times [45]^{41}$	4	48	256	128	128	64	25568
CEDRUS ^α -256f	32	65	14	8	45	$[18]^{63}$	2	63	256	128	128	64	43296
CEDRUS ^α -384s	48	65	8	12	38	$[22]^{70} \times [21]^{18}$	4	88	384	192	192	96	60672
CEDRUS ^α -384f	48	65	12	10	51	$[30]^{61} \times [29]^{19}$	2	80	384	192	192	96	76176
CEDRUS ^α -512s	64	65	8	13	47	$[36]^{49} \times [35]^{52}$	4	101	512	256	256	128	98048
CEDRUS ^α -512f	64	66	11	10	66	$[28]^{27} \times [27]^{82}$	4	109	512	256	256	128	127488

Table 1.2: Definition of the functions for CEDRUS^α-160 and CEDRUS^α-256

Function	Definition
$\mathbf{PRF}_{\text{MSG}}(\mathbf{SK}.\text{prf}, \text{optRand}, M)$	$\text{Trunc}_n(\text{SM3}(\mathbf{SK}.\text{prf} \parallel \text{optRand} \parallel M))$
$\mathbf{H}_{\text{MSG}}(\mathbf{R}, \mathbf{PK}.\text{seed}, \mathbf{PK}.\text{root}, M, \text{ctr})$	$\text{XOF}_{\text{SM3}}(\mathbf{R} \parallel \mathbf{PK}.\text{seed} \parallel \mathbf{PK}.\text{root} \parallel M \parallel \text{ctr}, 8m)$
$\mathbf{PRF}(\mathbf{PK}.\text{seed}, \mathbf{SK}.\text{seed}, \text{ADRS})$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel \mathbf{SK}.\text{seed}, 8n))$
$\mathbf{F}(\mathbf{PK}.\text{seed}, \text{ADRS}, M_1)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M_1, 8n))$
$\mathbf{H}(\mathbf{PK}.\text{seed}, \text{ADRS}, M_2)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M_2, 8n))$
$\mathbf{T}_l(\mathbf{PK}.\text{seed}, \text{ADRS}, M)$	$\text{Trunc}_n(\text{SM3}(\mathbf{PK}.\text{seed} \parallel \text{toByte}(0, 64 - n) \parallel \text{ADRS}^c \parallel M, 8n))$

Table 1.3: Definition of the functions for CEDRUS^α-384 CEDRUS^α-512

Function	Definition
$\mathbf{PRF}_{\text{MSG}}(\mathbf{SK.prf}, \text{optRand}, M)$	$\mathbf{XOF}_{\text{SM3}}(\mathbf{SK.prf} \parallel \text{optRand} \parallel M, 8n)$
$\mathbf{H}_{\text{MSG}}(\mathbf{R}, \mathbf{PK.seed}, \mathbf{PK.root}, M, \text{ctr})$	$\mathbf{XOF}_{\text{SM3}}(\mathbf{R} \parallel \mathbf{PK.seed} \parallel \mathbf{PK.root} \parallel M \parallel \text{ctr}, 8m)$
$\mathbf{PRF}(\mathbf{PK.seed}, \mathbf{SK.seed}, \text{ADRS})$	$\mathbf{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel \mathbf{SK.seed}, 8n)$
$\mathbf{F}(\mathbf{PK.seed}, \text{ADRS}, M_1)$	$\mathbf{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M_1, 8n)$
$\mathbf{H}(\mathbf{PK.seed}, \text{ADRS}, M_2)$	$\mathbf{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M_2, 8n)$
$\mathbf{T}_l(\mathbf{PK.seed}, \text{ADRS}, M)$	$\mathbf{XOF}_{\text{SM3}}(\mathbf{PK.seed} \parallel \text{ADRS} \parallel M, 8n)$

Chapter 2

Design Rationale

The SPHINCS⁺ framework provides the underlying architecture for modern quantum resistant stateless hash-based signatures. Notable examples include the NIST standard SLH-DSA [5] and its recent variants [3, 6, 2, 7]. We extend the hypertree structure that underlies the SPHINCS⁺ framework by allowing trees of different heights to appear on different layers, and we plug generalized hash-based one-time signatures with chains of different lengths into the hypertree. Moreover, we optimize the few time signatures based on the forced pruning technique. These optimizations lead to enlarged design space, opening up the possibility for finer-grained trade-offs. This chapter gives the rationale behind the building blocks (including the inhomogeneous WOTS^c one-time signature), and the CEDRUS^α hypertree framework.

2.1 Notations and Preliminaries

For a finite set $\mathbb{G}$, the number of elements in $\mathbb{G}$ is denoted by $|\mathbb{G}|$. We use $\mathbb{Z}_+$ to denote the set of all positive integers and let $\mathbb{N} = \mathbb{Z}_+ \cup \{0\}$. The set of real numbers is denoted by $\mathbb{R}$. Let $\mathbb{F}_2 = \{0, 1\}$ be the binary field and $\mathbb{F}_2^{8n}$ be the set of all n -byte strings. For a positive integer w , $[w]$ is defined to be the set $\{0, \dots, w-1\}$. The Cartesian product of two sets $\mathbb{A}$ and $\mathbb{B}$ is represented by $\mathbb{A} \times \mathbb{B}$. For $\mathbf{w} = (w_0, \dots, w_{l-1}) \in \mathbb{Z}_+^l$, the set $[w_0] \times \dots \times [w_{l-1}]$ is abbreviated as $\prod_{j=0}^{l-1} [w_j]$. When $w_0 = \dots = w_{l-1} = w$, $\prod_{j=0}^{l-1} [w_j] = [w]^l$. For $i \in \mathbb{N}$ and $m \in \mathbb{Z}_+$, $\text{Bin}(i, m)$ is the m -bit binary representation of i . For example, $\text{Bin}(5, 8) = 00000101$. Let x be a binary string, $\text{Int}(x)$ denotes the unsigned integer it represents.

Definition 1. Let $\mathbf{a} = (a_0, \dots, a_{l-1})$ and $\mathbf{b} = (b_0, \dots, b_{l-1})$ be elements in $\mathbb{N}^l$. Then, $\mathbf{a}$ and $\mathbf{b}$ are comparable if $a_i \leq b_i$ for all $i \in \{0, \dots, l-1\}$ or $b_i \leq a_i$ for all $i \in \{0, \dots, l-1\}$. A set $\mathbb{A} \subseteq \mathbb{N}^l$ is incomparable if $|\mathbb{A}| = 1$ or any two different elements in $\mathbb{A}$ are incomparable and a function $f : \mathbb{F}_2^{8n} \rightarrow \mathbb{B} \subseteq \mathbb{N}^l$ is incomparable if $f(\mathbb{F}_2^{8n}) \subseteq \mathbb{B}$ is incomparable.

Definition 2. A set $\mathbb{D} \subseteq \mathbb{R}^n$ is convex if for any $(x, y) \in \mathbb{D} \times \mathbb{D}$ and $0 \leq \lambda \leq 1$, $\lambda x + (1-\lambda)y \in \mathbb{D}$. Let $f : \mathbb{D} \rightarrow \mathbb{R}$ be a real-valued function defined over a convex set $\mathbb{D}$. f is called a convex function if for any $(x, y) \in \mathbb{D} \times \mathbb{D}$ and $0 \leq \lambda \leq 1$, $f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y)$.

Theorem 1 (Jensen's Inequality). Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a convex function. For any $(x_0, \dots, x_{d-1}) \in \mathbb{R}^d$, and $(\lambda_0, \dots, \lambda_{d-1}) \in \mathbb{R}_+^d$ with $\sum_{i=0}^{d-1} \lambda_i = 1$,

$$f\left(\sum_{i=0}^{d-1} \lambda_i x_i\right) \leq \sum_{i=0}^{d-1} \lambda_i f(x_i),$$

and the equality holds if and only if $x_0 = \dots = x_{d-1}$.

Lemma 2. Let $x_0, \dots, x_{d-1}$ be positive integers with $x_0 \leq x_1 \leq \dots \leq x_{d-1}$ and $x_0 + \dots + x_{d-1} = h$. If $|x_i - x_j| \leq 1$ for any $(i, j) \in [d] \times [d]$, then $x_0 = \dots = x_{d-k-1} = \lfloor \frac{h}{d} \rfloor$ and $x_{d-k} = \dots = x_{d-1} = \lceil \frac{h}{d} \rceil$, where $k = h - d \cdot \lfloor \frac{h}{d} \rfloor$.

Proof. See [2]. □

Definition 3. For given positive integers A and l , we define the notation

$$\left\langle \frac{A}{l} \right\rangle = \min_{\substack{(x_0, \dots, x_{l-1}) \in \mathbb{Z}_+^l \\ x_0 + \dots + x_{l-1} = A}} \sum_{i=0}^{l-1} 2^{x_i}.$$

Lemma 3. For given positive integers A and l ,

$$\left\langle \frac{A}{l} \right\rangle = \min_{\substack{(x_0, \dots, x_{l-1}) \in \mathbb{Z}_+^l \\ x_0 + \dots + x_{l-1} = A}} \sum_{i=0}^{l-1} 2^{x_i} = (l - k) \cdot 2^{\lfloor A/l \rfloor} + k \cdot 2^{\lceil A/l \rceil},$$

where $k = A - l \cdot \lfloor \frac{A}{l} \rfloor$. That is, $\sum_{i=0}^{l-1} 2^{x_i}$ reaches its minimum when $(l - k)$ x_i 's take on the value $\lfloor \frac{A}{l} \rfloor$, and k x_i 's take on the value $\lceil \frac{A}{l} \rceil$.

Proof. See [2]. □

2.2 The Inhomogeneous WOTS^α One-time Signature

Since in our framework, messages to be signed by a one-time signature scheme are typically the roots of certain hash trees, we restrict the message space of a one-time signature to be $\mathbb{B}^n$, i.e., the set of all n -byte strings. In addition, we present the construction using hash functions modeled as random oracles. We note, however, that modern hash-based one-time signatures rely on the so-called tweakable hash functions, and we expect all of our results to naturally carry over to that setting as well.

Let $H : \mathbb{F}_2^* \rightarrow \mathbb{F}_2^{8n}$ be a random oracle, $\text{len} \in \mathbb{Z}_+$, and $\mathbf{w} = (w_0, \dots, w_{\text{len}-1}) \in \mathbb{Z}_+^{\text{len}}$, such that $C_s \left(\prod_{j=0}^{\text{len}-1} [w_j] \right) \geq |\mathbb{F}_2^{8n}|$, where $s = \left\lfloor \frac{\sum_{i=0}^{\text{len}-1} (w_i - 1)}{2} \right\rfloor$. Then, we can construct a one-time signature scheme

$$\text{OTS}[H, \mathbb{F}_2^{8n}, \text{len}, \mathbf{w} = (w_0, \dots, w_{\text{len}-1})] = (\text{Gen}, \text{Sign}, \text{Ver})$$

as follows the following construction.

Construction 1. $\Phi = \text{OTS}[H, \mathbb{F}_2^{8n}, \text{len}, \mathbf{w} = (w_0, \dots, w_{\text{len}-1})] = (\text{Gen}, \text{Sign}, \text{Ver})$ is a one-time signature scheme with message space $\mathbb{F}_2^{8n}$ such that

- $(\text{sk}, \text{pk}) \leftarrow \text{Gen}(\mathbb{F}_2^{8n})$:
 - For $i \in [\text{len}]$: $\text{sk}_i \xleftarrow{\$} \mathbb{F}_2^{8n}$, $\text{pk}_i \leftarrow H^{w_i-1}(\text{sk}_i)$
 - $\text{sk} := (\text{sk}_0, \dots, \text{sk}_{\text{len}-1})$, $\text{pk} := (\text{pk}_0, \dots, \text{pk}_{\text{len}-1})$
- $\sigma \leftarrow \text{Sign}(\text{sk}, m)$:

- $x := (x_0, \dots, x_{\text{len}-1}) \leftarrow \rho_{\mathbf{w}}^s(\text{Int}(\mathbf{m}))$
- For $i \in [\text{len}]$: $\sigma_i \leftarrow H^{x_i}(\mathbf{sk}_i)$
- $\sigma := (\sigma_0, \dots, \sigma_{\text{len}-1}) \in [w_0] \times \dots \times [w_{\text{len}-1}]$
- $b \leftarrow \text{Ver}(\mathbf{pk}, \mathbf{m}, \sigma)$:
 - $(y_0, \dots, y_{\text{len}-1}) \leftarrow \sigma$
 - $x = (x_0, \dots, x_{\text{len}-1}) \leftarrow \rho_{\mathbf{w}}^s(\text{Int}(\mathbf{m}))$
 - For $i \in [\text{len}]$: $\mathbf{pk}'_i \leftarrow H^{w_i-1-x_i}(y_i)$
 - $b := (\mathbf{pk}_0 = \mathbf{pk}'_0) \wedge \dots \wedge (\mathbf{pk}_{\text{len}-1} = \mathbf{pk}'_{\text{len}-1})$

Note that in existing one-time signatures employed in hash-based signature schemes [5, 6, 8, 3], $w_0 = \dots = w_{l-1} = w$, that is, the virtual hash chains produced by the schemes are of the same length. However, we emphasize that the security proofs for one-time signature schemes based on hash chains with the same length naturally carry over to the general case where $(w_0, \dots, w_{l-1})$ can take any value in $\mathbb{Z}_+^{\text{len}}$. Allowing chains with different lengths enlarges the design space, potentially leading to improved signature size, signing and verification performance.

Let $\mathfrak{C}_{\text{Sign}}(\Phi)$, $\mathfrak{C}_{\text{Ver}}(\Phi)$, and $\mathfrak{C}_{\text{Size}}(\Phi)$ denote the signing cost, verification cost, and signature size of Φ , respectively. According to Construction 1, we have the following estimation

$$\begin{cases} \mathfrak{C}_{\text{Size}}(\Phi) = \text{len} \cdot n \\ \mathfrak{C}_{\text{Sign}}(\Phi) = \left\lfloor \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rfloor + \text{len} \\ \mathfrak{C}_{\text{Ver}}(\Phi) = \left\lceil \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rceil \end{cases}, \quad (2.1)$$

where the unit for $\mathfrak{C}_{\text{Sign}}(\Psi)$ and $\mathfrak{C}_{\text{Ver}}(\Psi)$ is the number of hash function evaluations and the unit for $\mathfrak{C}_{\text{Size}}(\Psi)$ is the number of bytes.

2.3 The CEDRUS^α Hypertree

The structure of the CEDRUS^α virtual hypertree is described in Section 1.4. In our scheme, the height of the XMSS trees in layer i of the d -layer hypertree is h_i , while in existing hash-based signature schemes, $h_0 = \dots = h_{d-1}$. In short, we allow trees in different layers to be of different heights (but trees in the same layer are of the same height), which enlarges the design space. Let $\mathfrak{C}_{\text{Sign}}(\Psi)$, $\mathfrak{C}_{\text{Ver}}(\Psi)$, and $\mathfrak{C}_{\text{Size}}(\Psi)$ denotes the signing cost, verification cost, and signature size of Ψ respectively for the hypertree signature described in Section 1.8. Then, we have the following estimation

$$\begin{cases} \mathfrak{C}_{\text{Size}}(\Psi) = d \cdot \text{len} \cdot n + h \cdot n \\ \mathfrak{C}_{\text{Sign}}(\Psi) = d \cdot \left(\left\lfloor \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rfloor + \text{len} \right) + (2^{h_0} - 1) + \dots + (2^{h_{d-1}} - 1) \\ \mathfrak{C}_{\text{Ver}}(\Psi) = d \cdot \left(\left\lceil \frac{(w_0-1) + \dots + (w_{\text{len}-1}-1)}{2} \right\rceil \right) + h \end{cases}, \quad (2.2)$$

where the unit for $\mathfrak{C}_{\text{Sign}}(\Psi)$ and $\mathfrak{C}_{\text{Ver}}(\Psi)$ is the number of hash function evaluations and the unit for $\mathfrak{C}_{\text{Size}}(\Psi)$ is the number of bytes. For a given value of $(\text{len}, \mathbf{w}, h, d, n)$, according to Lemma 3 and Equation (2.2), setting $h_0 = \dots = h_{k-1} = \lceil \frac{h}{d} \rceil$ and $h_k = \dots = h_{d-1} = \lfloor \frac{h}{d} \rfloor$ minimizes $\mathfrak{C}_{\text{Sign}}(\Psi)$, i.e., there is in essence a unique optimal value for $(h_0, \dots, h_{d-1})$. Note that we choose $h_0 = \dots = h_{k-1} = \lceil \frac{h}{d} \rceil$ rather $h_0 = \dots = h_{k-1} = \lfloor \frac{h}{d} \rfloor$ since it leads to better performance for CEDRUS^α key generation.

Chapter 3

Security Analysis

3.1 Security Reduction

Since the one-time signature scheme and the **XMSS** trees with different heights do not affect the security proof given in [6], we refer the reader to [6] for the security reduction.

3.2 Concrete Security

We focus on our security analysis on the case where an attacker forges a **CEDRUS**^α signature by forging a **FORC** few-time signature for a unsigned message based on the information released as part of many **CEDRUS**^α signatures issued previously without computing a (2nd) preimage of the underlying hash functions. Other strategies of generating a forgery are either covered by the existing analysis of the **SPHINCS**⁺ framework or have a trivial complexity of about $\mathcal{O}(2^{8n})$.

Let q_{sig} be the number of signing queries overall. Then, the probability that a given **FORC** instance is used exactly q times is about

$$\phi(q) = \binom{q_{\text{sig}}}{q} \left(1 - \frac{1}{2^h}\right)^{q_{\text{sig}}-q} \left(\frac{1}{2^h}\right)^q. \quad (3.1)$$

Let $p_{\text{Succ}}^{\text{FTS}}(q)$ be the probability that any given q signing queries of a specific **FORC** instance can be used to successfully produce a forgery. Then, the probability $p_{\text{Succ}}(q_{\text{sig}})$ of successfully producing a **CEDRUS**^α signature with q_{sig} signing queries is about

$$p_{\text{Succ}}(q_{\text{sig}}) = \sum_{q=0}^{q_{\text{sig}}} \binom{q_{\text{sig}}}{q} \left(1 - \frac{1}{2^h}\right)^{q_{\text{sig}}-q} \left(\frac{1}{2^h}\right)^q p_{\text{Succ}}^{\text{FTS}}(q), \quad (3.2)$$

where $p_{\text{Succ}}^{\text{FTS}}(q) = \left(1 - \left(1 - \frac{1}{2^a} \cdot \frac{w'+1}{2^{w'}}\right)^q\right)^k$.

Chapter 4

Performance Analysis

In this chapter, we analyze the performance of our design, implement instances with the parameter sets defined in Section 1.11, and report on benchmark results.

4.1 Theoretical Performance Analysis

A CEDRUS^α signature contains an n -byte randomizer, an $k \cdot (a + 1) \cdot n$ -byte FORC signature, $d \cdot \text{len} \cdot n$ -byte WOTS^α signatures, and h n -byte authentication nodes. Therefore, the total size of a CEDRUS^α signature is

$$n + k \cdot (a + 1) \cdot n + d \cdot \text{len} \cdot n + h \cdot n \quad (4.1)$$

bytes in length. Next, we analyze the performances of key generation, signing, and verification in terms of the number of hash function calls.

Key Generation Performance. The complexity of key generation is dominated by the complexity of computing the root of the top-level XMSS tree (Line 3 of Algorithm 19). Let $\mathfrak{C}_{\text{xmssNode}}(z)$ be the number of hash function evaluations of

$$\text{xmssNode}(\mathbf{SK}.\text{seed}, i, z, \mathbf{PK}.\text{seed}, \text{ADRS}).$$

Then, according to Algorithm 8, we have

$$\begin{cases} \mathfrak{C}_{\text{xmssNode}}(z) = 2\mathfrak{C}_{\text{xmssNode}}(z - 1) + 1 \\ \mathfrak{C}_{\text{xmssNode}}(0) = \text{len} + (w_0 - 1) + \dots + (w_{\text{len}-1} - 1) + 1 = w_0 + \dots + w_{\text{len}-1} + 1 \end{cases} \quad (4.2)$$

Solving the recurrence given by Equation (4.2) gives $\mathfrak{C}_{\text{xmssNode}}(z) = 2^z \cdot \mathfrak{C}_{\text{xmssNode}}(0) + 2^z - 1$. Therefore, it requires about

$$(w_0 + \dots + w_{\text{len}-1} + 2) \cdot 2^{h_d-1} - 1 \quad (4.3)$$

hash function evaluations.

Signing Performance. The CEDRUS^α signing process involves one hash function call to compute the initial **digest**, one FORC signing operation, one call to $\text{pkFromSig}_{\text{FORC}}()$ to compute the root of the FORC tree (Algorithm 18), d WOTS^α signing operations, the computation

of h authentication nodes, and $d - 1$ calls to $\text{pkFromSig}_{\text{XMSS}}()$ to compute the roots of the XMSS trees (Algorithm 10), requiring h hash evaluations in total.

Let us first investigate the complexity of one FORS signing operation. It includes k hash evaluations for generating the k secret n -byte elements, $k \cdot (\mathfrak{C}_{\text{FTSNode}}(0) + \dots + \mathfrak{C}_{\text{FTSNode}}(a-1)) = k \cdot (3 \cdot 2^a - a - 3)$ hash evaluations to compute the authentication nodes, where $\mathfrak{C}_{\text{FTSNode}}(0) = 2$, and $\mathfrak{C}_{\text{FTSNode}}(z) = 2\mathfrak{C}_{\text{FTSNode}}(z-1) + 1$. In short,

$$\mathfrak{C}_{\text{FTS}}(t, k) = k + k \cdot (3 \cdot 2^a - a - 3)$$

To generate one WOTS $^\alpha$ signature, it takes about

$$\mathfrak{C}_{\text{Sig}_{\text{WOTS}}}(\mathbf{w}, \text{len}) = \left\lfloor \frac{(w_0 - 1) + \dots + (w_{\text{len}-1} - 1)}{2} \right\rfloor + \text{len} \quad (4.4)$$

hash evaluations. Let $\mathfrak{C}_{\text{xmssAuth}}(h_i)$ be the number of hash evaluations required to compute the authentication nodes of an XMSS tree with height h_i . Then,

$$\begin{aligned} \mathfrak{C}_{\text{xmssAuth}}(h_i) &= \mathfrak{C}_{\text{xmssNode}}(0) + \dots + \mathfrak{C}_{\text{xmssNode}}(h_i - 1) \\ &= (\mathfrak{C}_{\text{xmssNode}}(0) \cdot 2^0 + 2^0 - 1) + \dots + (\mathfrak{C}_{\text{xmssNode}}(0) \cdot 2^{h_i-1} + 2^{h_i-1} - 1) \\ &= \mathfrak{C}_{\text{xmssNode}}(0) \cdot (2^{h_i} - 1) + (2^{h_i} - 1) - h_i. \end{aligned} \quad (4.5)$$

Consequently, the signing process takes about

$$1 + \mathfrak{C}_{\text{FTS}}(t, k) + d \cdot \mathfrak{C}_{\text{Sig}_{\text{WOTS}}}(\mathbf{w}, \text{len}) + \sum_{i=0}^{d-1} \mathfrak{C}_{\text{xmssAuth}}(h_i) + h,$$

which is

$$1 + \mathfrak{C}_{\text{FTS}}(t, k) + d \cdot \mathfrak{C}_{\text{Sig}_{\text{WOTS}}}(\mathbf{w}, \text{len}) + (\mathfrak{C}_{\text{xmssNode}}(0) + 1) \left(\left(\sum_{i=0}^{d-1} 2^{h_i} \right) - d \right)$$

hash evaluations to generate one CEDRUS $^\alpha$ signature.

Verification Performance. Verification involves one hash call to compute the initial digest. One call to $\text{pkFromSig}_{\text{PORS}}()$, which requires $k \cdot a + 1$ hash function calls. One call to $\text{verify}_{\text{HT}}()$, which requires $\left\lfloor \frac{(w_0 - 1) + \dots + (w_{l-1} - 1)}{2} \right\rfloor \cdot d + h$ hash function calls. It takes

$$1 + (k \cdot a + 1) + \left(\left\lfloor \frac{(w_0 - 1) + \dots + (w_{l-1} - 1)}{2} \right\rfloor \cdot d + h \right) \quad (4.6)$$

hash evaluations to verify one CEDRUS $^\alpha$ signature.

4.2 Implementations and Benchmarks

We implement the CEDRUS $^\alpha$ instances for the defined parameter sets given in Section 1.11, the theoretical analysis are listed in Table 4.1, and the benchmark resulting cycle counts are listed in Table 4.2. All of the results are obtained from running our codes compiled with `gcc-11.4.0-03-march=native-fomit-frame-pointer-flto` on a Ubuntu 22.04 machine with Intel Core i5-1135G7 CPU and 16GB RAM. It should be noted that, in the AVX2-optimized implementation, we did not optimize the hash function itself. Instead, we still use the reference hash implementation provided in the official `API.PKC.zip`.

Table 4.1: A theoretical analysis of the performance of the proposed signature algorithms

Scheme	PK Size (Bytes)	SK Size (Bytes)	Signature Size (Bytes)	Signing (#Hash call)	Verification (#Hash call)
CEDRUS ^{α} -160s	40	80	10300	2743319	6710
CEDRUS ^{α} -160f	40	80	19420	199011	6120
CEDRUS ^{α} -256s	64	128	25568	4163230	9975
CEDRUS ^{α} -256f	64	128	43296	456199	8019
CEDRUS ^{α} -384s	96	192	60672	5347900	7993
CEDRUS ^{α} -384f	96	192	76176	1490831	14489
CEDRUS ^{α} -512s	128	256	98048	10555995	14800
CEDRUS ^{α} -512f	128	256	127488	2481272	16731

Table 4.2: Benchmark results for the proposed signature algorithms

Scheme	Key Generation (cycles)	Signing (cycles)	Verification (cycles)
CEDRUS ^{α} -160s	334855475	4973413079	12237234
CEDRUS ^{α} -160f	11253560	365227595	11368588
CEDRUS ^{α} -256s	508682968	7738937102	18762819
CEDRUS ^{α} -256f	36556224	974104514	19860937
CEDRUS ^{α} -384s	2701027283	29222821110	45102874
CEDRUS ^{α} -384f	435231937	8580012818	83764473
CEDRUS ^{α} -512s	5499485228	57965845699	82986215
CEDRUS ^{α} -512f	1056815134	13883220327	97488988

Chapter 5

Feature Overview

In this chapter, we summarize the new techniques employed in our design for improving the cost and performance of stateless hash-based signature schemes. Also, we discuss the advantages and limitations of the CEDRUS^α hash-based signature scheme.

5.1 Techniques for Improving the SPHINCS^+ Framework

The SPHINCS^+ framework provides the underlying architecture for modern quantum resistant stateless hash-based signatures. Notable examples include the NIST standard SLH-DSA and its recent variants such as $\text{SPHINCS-}\alpha$ and $\text{SPHINCS}^+\text{C}$. We extend the hypertree structure that underlies the $\text{SPHINCS-}\alpha$ framework [3] by allowing trees of different heights to appear on different layers, and we plug generalized hash-based one-time signatures with chains of different lengths into the hypertree. These techniques lead to enlarged design space, opening up the possibility for finer-grained trade-offs.

The CEDRUS^α Hypertree. We further extend the hypertree structure of SPHINCS^+ and its variants by allowing the Merkle-like trees in different layers to have different heights. Let $h > 0$ be the total height and d be the number of layers such that $h = h_0 + \dots + h_{d-1}$, where $h_i > 0$ is the height of the trees in the i -th layer of the extended hypertree. For a given h , we show that in essence there is a unique choice of $(h_0, \dots, h_{d-1})$ that optimizes the performance.

One-time Signatures. We generalize the one-time signatures based on hash chains with chains of different lengths, and the resulting design is named as *inhomogeneous* hash-chain-based one-time signatures. In particular, we propose an *inhomogeneous* hash-chain-based one-time signature scheme based on WOTS^α . Simply plugging the inhomogeneous one-time signatures into SPHINCS^+ and its variants leads to size and signing and verification speed improvements.

5.2 Advantages and Limitations

The advantages and limitations of CEDRUS^α can be summarized as follows. It is widely regarded as one of the most conservative designs among post-quantum signature schemes, yet it comes with the inefficiencies in terms of signature size and speed. Below, we examine its

advantages and limitations in greater detail.

Minimal Security Assumptions. Unlike other post-quantum cryptographic schemes, including both signature and public-key encryption designs, CEDRUS^α does not rely on any newly introduced intractability assumption. Its security is based solely on well-studied assumptions regarding the underlying hash function. While any practical signature scheme supporting arbitrary-length input requires a secure hash function, CEDRUS^α offers an added layer of robustness: even a collision attack on the hash function does not compromise its security. This is a notable advantage in light of successful collision attacks against algorithms such as MD5 and SHA-1, especially considering that second-preimage resistance remains unbroken for MD5. Furthermore, the cryptographic community has developed a solid understanding of (exact) hash-function security, particularly following the recent SHA3 competition. This stands in contrast to the relatively novel mathematical problems employed in other branches of post-quantum cryptography. Although some of those problems have been known for a long time, accurately estimating the hardness of solving concrete instances remains far less mature.

Small key sizes. An additional advantage of CEDRUS^α lies in its compact key sizes, particularly that of the public key. In many practical applications where certificates or certificate chains are routinely exchanged. The small public key size of CEDRUS^α thus translates into tangible savings in bandwidth and storage in such deployment scenarios.

Signature size and speed. The primary drawback of CEDRUS^α lies in its signing speed and signature size: it is not designed to compete as the fastest or most compact signature scheme. Nevertheless, the framework offers a wide range of possible trade-offs, allowing parameters to be adjusted as long as at least one of the two limitations – moderately slow signing or moderately large signatures – can be accommodated. Moreover, CEDRUS^α is fairly efficient in signature verification, a feature that makes it well-suited for application scenarios such as secure software updates, where verification performance is often more critical than signing speed.

Bibliography

- [1] NICCS. Call for Proposals for the Next-generation Public-Key Cryptographic Algorithms, 2025. https://www.niccs.org.cn/niccs/Notice/pc/content/content_1975896137741635584.html.
- [2] Zhen Qin and Siwei Sun. Extending the SPHINCS⁺ framework: Varying the tree heights and chain lengths. Cryptology ePrint Archive, Paper 2025/2236, 2025. <https://eprint.iacr.org/2025/2236>.
- [3] Kaiyi Zhang, Hongrui Cui, and Yu Yu. Revisiting the Constant-Sum Winternitz One-Time Signature with Applications to SPHINCS⁺ and XMSS. In Helena Handschuh and Anna Lysyanskaya, editors, *Advances in Cryptology - CRYPTO 2023 - 43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 455–483. Springer, 2023.
- [4] Ray A. Perlner, John Kelsey, and David A. Cooper. Breaking category five SPHINCS⁺ with SHA-256. In Jung Hee Cheon and Thomas Johansson, editors, *Post-Quantum Cryptography - 13th International Workshop, PQCrypto 2022, Virtual Event, September 28-30, 2022, Proceedings*, volume 13512 of *Lecture Notes in Computer Science*, pages 501–522. Springer, 2022.
- [5] NIST. Stateless Hash-Based Digital Signature Standard. Federal Information Processing Standards Publication, FIPS 205, 2024. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.pdf>.
- [6] Andreas Hülsing, Mikhail A. Kudinov, Eyal Ronen, and Eylon Yogev. SPHINCS+C: compressing SPHINCS⁺ with (almost) no cost. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 1435–1453. IEEE, 2023.
- [7] Mehdi Abri and Jonathan Katz. Shorter hash-based signatures using forced pruning. Cryptology ePrint Archive, Paper 2025/2069, 2025. <https://eprint.iacr.org/2025/2069>.
- [8] David McGrew, Michael Curcio, and Scott Fluhrer. Leighton-Micali Hash-Based Signatures. RFC 8554, 2019. <https://datatracker.ietf.org/doc/html/rfc8554>.